



Highway to Renewable Energy Systems

H2RES v1.3 Report (Draft)

User Manual for Research, Scenario Design, and Model Deployment

Document Type: User Manual

Reviewed by:

Date: June 9, 2026

Contents

1	Purpose and Scope	3
1.1	What H2RES Solves	3
1.2	Intended Users	3
1.3	How to Use This Manual	3
1.4	Core Abbreviations and Nomenclature	4
1.5	Index Symbols Used Throughout the Manual	4
1.6	General Changes Introduced in v1.2	4
2	Recommended End-to-End Workflow	5
2.1	Operational Sequence	5
2.2	Execution Command	5
2.3	Runtime Notes	7
3	First Baseline Run	7
3.1	Pre-launch baseline checklist	7
3.2	Baseline execution procedure	7
3.3	What indicates a successful first run	8
3.4	Common first-run discipline	8
4	Scenario Design in main.py	8
4.1	Boolean Switches and Direct Effects	8
4.2	Time-Horizon and Rolling-Window Controls	9
4.3	National Policy Vectors and General Cost Controls	9
4.4	Transport and Flexibility Assumptions	9
4.5	Input Path Declarations in the Entrypoint	9
5	Input Data Engineering Guide	10
5.1	Why this is critical	10
5.2	Required Input Files and Roles	10
5.3	Interpreting the genco master table	12
5.4	How Wide-Format Files Become Long-Format Tables	14
5.5	Cross-file Consistency Rules	14
5.6	Pre-run Data Validation Checklist	16
6	How H2RES Represents the Energy System	17
6.1	Integrated system view	17
6.2	How input data becomes model structure	17
6.3	Electricity balance logic	17
6.4	Heat and cooling service logic	19
6.5	Hydrogen subsystem logic	19
6.6	Storage logic across carriers	20
7	Adapting H2RES to a New Case Study	21
7.1	What usually needs to be changed	21
7.2	Recommended order when adapting the model	22
7.3	What should usually <i>not</i> be changed first	22

8	Traceability: Input Data to Model Blocks	22
8.1	Block-Level Traceability Matrix	22
8.2	Critical Parameter Traceability Examples	23
8.3	Comprehensive Parameter Hand-off Table	23
8.4	Decision Variables Organized by Model Block	26
8.5	Unit Convention and Dimensional Consistency	26
8.6	Embedded Model Traceability Within This Manual	27
9	Interpreting Outputs	27
9.1	Where results are written	27
9.2	Main output families	27
9.3	Recommended reading order for outputs	28
10	Troubleshooting Matrix	28
11	Verification and Validation (V&V) Protocol	29
11.1	Structural Validation Before Solve	29
11.2	Numerical and Behavioral Validation After Solve	30
11.3	Scenario Toggle Regression Tests	31
12	PhD-Level Research Practice and Reproducibility	31
12.1	Minimum reproducibility package per scenario	31
12.2	Reproducible Baseline Mini-Case (Bundled Multizone Template)	31
12.3	Recommended experimental protocol	32
12.4	When to modify equations	32
13	Modeling Assumptions, Implications, and Risks by Block	32
14	Known Boundaries and Open Items	33
A	Flex-Technology Workbook Sheet Roles	34
B	Suggested Future Validation Script Logic	34
C	Equation Traceability from Build_model.py with User Commentary	34
C.1	Code-to-Equation Mapping (selected core constraints)	35
C.2	How to Read This Traceability Appendix	35
D	Complete Data Matching and Consistency Map	36
D.1	Hard Joins in Code	36
D.2	Required Schema Columns and Identifier Fields	37
D.3	Semantic Consistency Checks That May Not Trigger Immediate Errors	38
D.4	Recommended Pre-Run Review Logic	39

1 Purpose and Scope

This manual is intended to support rigorous use, adaptation, and extension of H2RES in research and applied energy-system analysis. It complements the Mathematical Formulation Report.

This document focuses on:

- End-to-end user workflow from scenario design to result interpretation.
- Input data engineering and cross-file consistency.
- Practical traceability between input files, internal parameters, and model blocks.
- Troubleshooting patterns observed from the current repository structure.
- Research-grade reproducibility practices for future case studies.

1.1 What H2RES Solves

H2RES is an hourly energy system optimization model that co-optimizes:

- Electricity generation and capacity investment.
- Intertemporal storage (hydro, stationary electrochemical storage, EV fleet storage).
- Sector coupling with heat, cooling, and hydrogen pathways.
- Policy instruments such as RPS trajectories, carbon caps, and optional reserve modules.

1.2 Intended Users

- **New users:** run the bundled multizone reference workflow and understand outputs.
- **Researchers:** prepare new case-study datasets and scenario assumptions.
- **Developers:** modify equations after validating data and workflow consistency.

1.3 How to Use This Manual

Reading objective	Start here	What this part gives you
Understand what H2RES represents	Sections 1 and 6	System scope, carrier interactions, and the physical meaning of the main subsystems before touching input files or code.
Run the model for the first time	Sections 2 and 3	Execution order, first-run checks, and the minimum evidence that a baseline run completed correctly.
Modify scenario assumptions	Sections 4 and 5	Which settings belong in <code>main.py</code> , which files control data, and how cross-file consistency must be preserved.
Adapt the model to a new case study	Sections 7 and 8	What to modify, in what order, and how those changes propagate into model blocks and internal parameters.
Interpret and validate results	Sections 9 to 12	Output reading order, troubleshooting, validation logic, and reproducibility expectations.

1.4 Core Abbreviations and Nomenclature

This table consolidates the abbreviations that recur most often in the user manual and in the Mathematical Formulation Report.

Abbreviation	Expanded term	Practical meaning in H2RES
CEEP	Critical excess / curtailed electric-energy placeholder	Slack-like adequacy variable used when the electricity balance cannot be satisfied strictly.
CHP	Combined heat and power	Units that jointly produce electricity and useful heat under coupling constraints.
COP	Coefficient of performance	Heat-pump conversion ratio from electricity input to heat or cooling output.
DH	District heating	Aggregated thermal markets linked to CHP units, heat pumps, and boilers.
EV	Electric vehicle	Aggregate fleet-storage representation used for transport flexibility.
FC	Fuel cell	Hydrogen-to-electricity conversion technology.
H2	Hydrogen	Energy carrier modeled through electrolysis, storage, direct demand, and fuel-cell use.
HDAM	Hydro dam reservoir	Reservoir hydro unit with intertemporal storage dynamics.
HP	Heat pump	Electricity-driven heat or cooling technology in DH or general-demand blocks.
HPHS	Pumped-hydro storage	Hydro storage technology with intertemporal water inventory behavior.
HROR	Hydro run-of-river	Non-dispatchable hydro technology without reservoir optimization.
MILP/LP	Mixed-integer / linear program	Optimization model class used depending on active modules and formulation choices.
NPV	Net present value factor	Intertemporal discounting weight applied in the objective.
NTC	Net transfer capacity	Import-capacity ceiling used in electricity-balance logic.
RPS	Renewable portfolio standard	Policy block enforcing a minimum renewable generation share.
SOC	State of charge	Intertemporal inventory state for EV, battery, hydro, heat, or hydrogen storage.

1.5 Index Symbols Used Throughout the Manual

The mathematical and traceability sections rely on a compact set of recurring indices. Listing them in one place reduces the need to cross-check the Mathematical Formulation Report while reading the manual.

Index	Typical set or object	Meaning in H2RES
<i>u</i>	units	Electricity-generation unit in the main plant registry.
<i>t</i>	periods	Hourly or time-slice period within a model year.
<i>y</i>	years	Model year in the planning horizon.
<i>b</i>	storage or boiler subsets	Stationary battery unit or, depending on the block, thermal boiler index.
<i>h</i>	HeatPump_units	Heat-pump technology or market-linked heat-pump identifier.
<i>m</i>	CHP or DH-market subsets	CHP-linked heat market or district-heating market identifier.
<i>s</i>	demand or storage subsets	Demand sector or hydrogen-storage index, depending on the equation block.
<i>e</i>	Elec_h2_units	Electrolyzer identifier.
<i>c</i>	FC_units	Fuel-cell identifier.
<i>f</i>	fuel subsets	Fuel type, especially in industry or emissions blocks.
<i>q</i>	hydrogen-demand subsets	Hydrogen-demand sector or use category.

1.6 General Changes Introduced in v1.2

The current v1.2 implementation changes the way H2RES is launched and managed operationally, even though the underlying energy-system structure remains recognizable from v1.1.

- The main entrypoint is now `main_rolling_horizon_v3.py`, which replaces the older single-file launch pattern centered on `main.py`.
- A new boolean control, `Rolling_Horizon`, determines whether the model is solved as a standard full-horizon problem or through a sequential rolling-horizon workflow.
- The execution layer now routes through `scripts/modeling_approaches.py`, which dispatches either the standard build/export path or the rolling-horizon path based on the selected mode.

- When the rolling-horizon mode is active, H2RES solves a sequence of overlapping subproblems over a user-defined planning window, rolls forward to the next window, and carries cumulative investment decisions into the next solve.
- Time-horizon controls are now exposed explicitly through `periods`, `years`, and `window`, so temporal scope is configured directly in the entrypoint rather than being fixed deeper in the repository.
- The active v1.2 data package is organized as a scenario subpackage under `data/SC`, and the rolling-horizon mode exports results through `export_plot_RH.py` while preserving the same top-level results-folder logic.

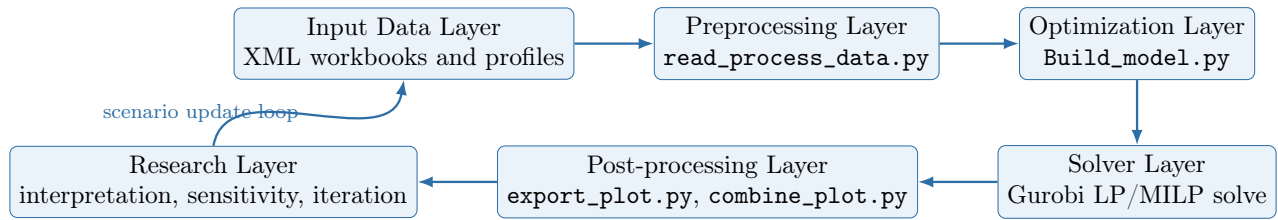


Figure 1: Layered architecture of H2RES from data engineering to research decisions.

2 Recommended End-to-End Workflow

This section is operational. It describes the sequence a user follows to run the current v1.3 repository from repository root to interpreted multizone results.

2.1 Operational Sequence

The current `main.py` follows this execution sequence:

1. `scripts/read_process_data.py`
2. `scripts/modeling_approaches.py`
3. If `Rolling_Horizon=True`, the wrapper iterates planning windows internally and repeatedly calls `Build_model.py`.
4. The rolling-horizon exporter `export_plot_multizone_RH.py` reconstructs the consolidated multizone output package.
5. The auxiliary utility `save_all_results_to_csv.py` writes the internal result dictionaries to `results/<scen>/data_result/`.

2.2 Execution Command

Run command: `python main.py`

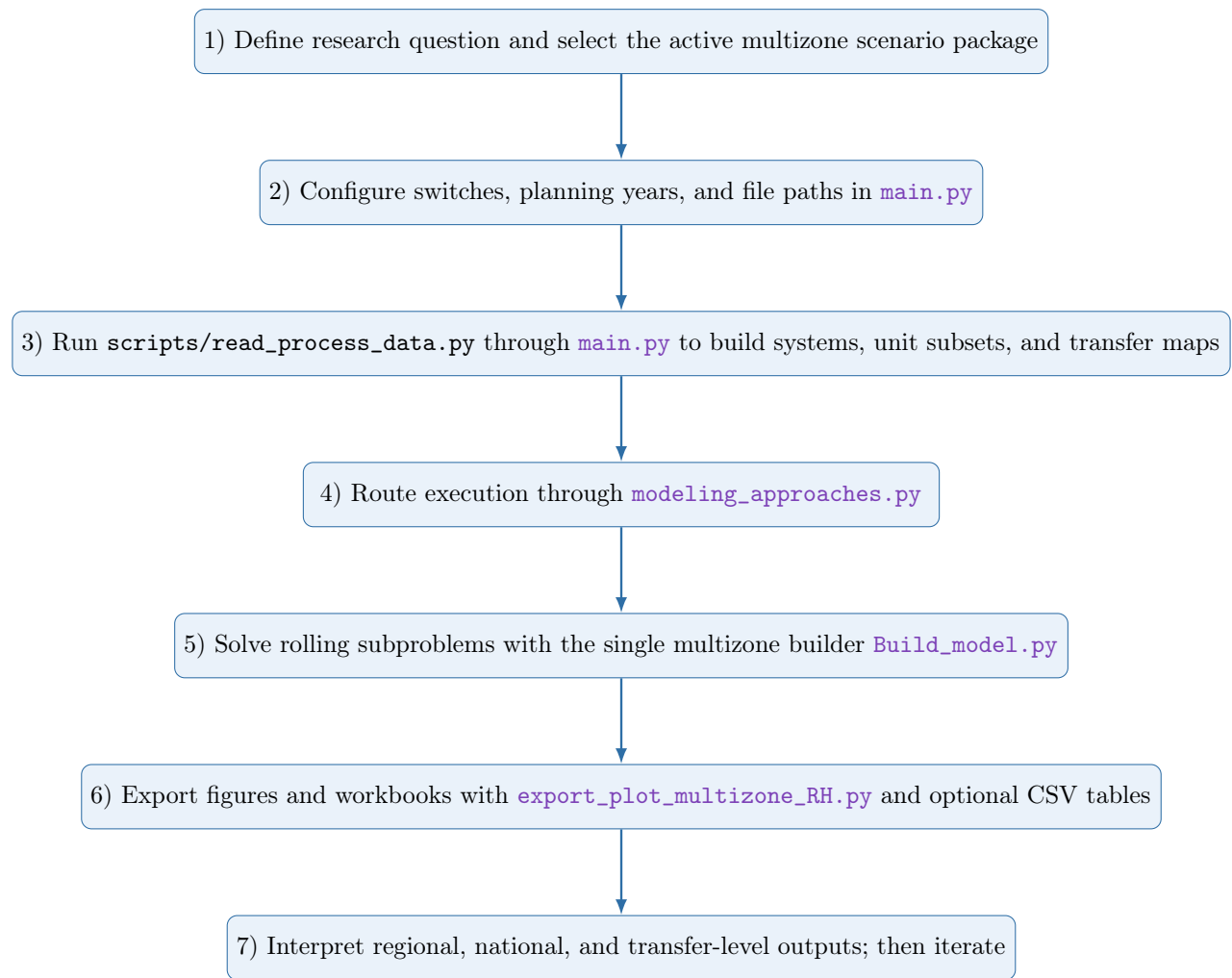


Figure 2: User workflow aligned with the active v1.3 multizone rolling-horizon execution path.

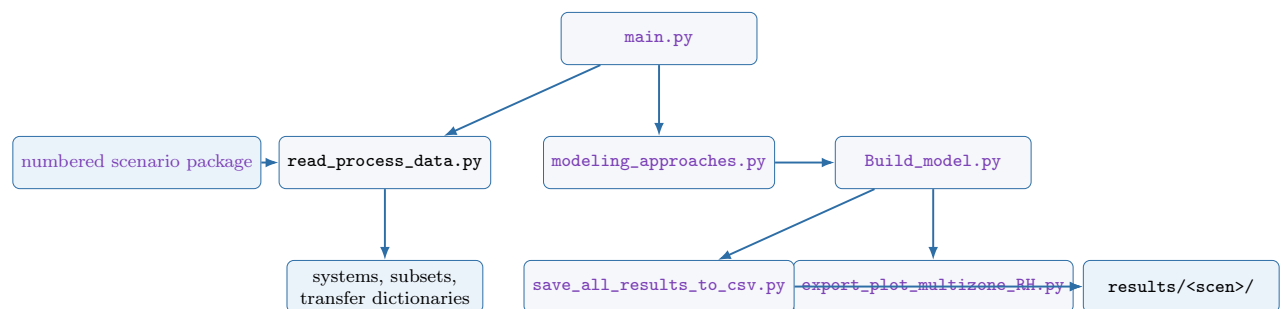


Figure 3: Execution dependency map of the active v1.3 script stack.

2.3 Runtime Notes

- The run should be launched from repository root so all relative paths resolve.
- In the current v1.3 folder snapshot, the complete bundled workflow is the multizone rolling-horizon path. Keep `Rolling_Horizon=True` unless the legacy full-horizon export scripts are restored intentionally.
- `export_plot_multizone_RH.py` removes and recreates `./results/<scenario>/`; archive old runs before reusing a scenario name.
- The repository still depends on a correctly configured Python/Gurobi environment outside the LaTeX documentation itself.

3 First Baseline Run

For a new user, the first target should be a clean multizone baseline run with no equation changes and no schema changes in the numbered input package. The purpose of this first pass is to confirm that the environment, regional identifiers, transfer definitions, active scenario switches, and rolling-horizon export logic are behaving as expected before any scientific interpretation begins.

3.1 Pre-launch baseline checklist

- Confirm that the run is launched from repository root so all relative paths resolve correctly.
- Open `main.py` and record the active scenario name `scen`, key boolean switches, the selected multizone package, and the active national-policy controls.
- Keep `Rolling_Horizon=True` for the bundled v1.3 workflow unless you have restored the legacy full-horizon export branch yourself.
- Confirm that all mandatory numbered input files referenced in Section 5 are present in the expected data folder.
- Archive any old folder with the same scenario name, because the active exporter recreates the result directory.
- Do not modify equations, identifiers, or workbook schemas before the baseline run succeeds once.

3.2 Baseline execution procedure

1. Open `main.py` and review only the scenario-level assumptions and file paths required for the first run.
2. Run `python main.py` from repository root.
3. Allow the preprocessing stage to build `systems`, `unit_system`, demand dictionaries, and transfer-link dictionaries from `1_system_data.xlsx` and the companion files.
4. Check that the console reports rolling-window solution messages if `Rolling_Horizon=True`.
5. Confirm that a new folder is created under `results/<scenario_name>/`.
6. If the CSV dump utility is active, inspect `results/<scenario_name>/data_result/` as the lowest-level exported trace of model variables.

3.3 What indicates a successful first run

Checkpoint	Expected sign	Why it matters
Preprocessing stage	No missing-key, missing-sheet, or mismatch error during data loading	Confirms that region names, unit identifiers, transfer links, and time axes are consistent enough to build model dictionaries.
Optimization stage	Rolling subproblem messages followed by feasible/optimal completion status	Confirms that the multizone package and scenario assumptions are numerically consistent.
Results folder	<code>results/<scenario_name>/</code> is recreated and populated	Confirms that the export stack ran and wrote outputs to the expected scenario directory.
CSV trace layer	<code>results/<scenario_name>/data_result/</code> exists when the CSV utility runs	Provides a low-level audit trail for regional variables, transfer flows, and investment decisions.

3.4 Common first-run discipline

For the first successful run, change as little as possible. If the baseline fails, do not move immediately to new policies, new technologies, or new datasets. First establish whether the issue comes from environment setup, identifier matching, transfer topology, time indexing, or scenario-switch logic.

4 Scenario Design in `main.py`

4.1 Boolean Switches and Direct Effects

Parameter	Type	Default	User-level meaning	Effect on model behavior
<code>rps_inv</code>	bool	False	Activate regional renewable-share policy logic.	If False, only the national renewable-share layer can remain active.
<code>carbonLimit</code>	bool	False	Activate regional annual CO2-cap logic.	If False, only the national CO2-cap layer can remain active.
<code>save_csv</code>	bool	False	Export workbook outputs from the plotting/export layer.	If False, figures still exist but optional workbook writing is reduced.
<code>res_inv</code>	bool	True	Allow renewable and flexibility investment pathways.	If False, relevant investment branches are forced to zero in the builder.
<code>hydro_storage</code>	bool	True	Activate HDAM/HPHS intertemporal storage dynamics.	If False, reservoir recursion constraints are skipped.
<code>ceep_limit</code>	bool	True	Enforce the annual CEEP share bound.	If False, the annual adequacy-relaxation cap is removed.
<code>NoResToHeatInv</code>	bool	False	Disable the power-to-heat route.	If True, heat-pump investment and electricity-to-heat conversion are forced to zero.
<code>Rolling_Horizon</code>	bool	True	Use the bundled rolling-horizon orchestration path.	If True, <code>modeling_approaches.py</code> iterates planning windows while repeatedly calling the single multi-zone builder.
<code>Transmission_Inv</code>	bool	False	Allow endogenous reinforcement of inter-system transfer capacity.	If False, all <code>transfer_inv</code> decisions are forced to zero.
<code>rps_inv_National</code>	bool	True	Activate a national renewable-share requirement across all systems.	Adds a cross-system RPS constraint on aggregated generation.
<code>carbonLimit_National</code>	bool	True	Activate a national CO2 cap across all systems.	Adds a national emissions constraint on the sum of regional emissions.

The commented switches `ceep_penalty` and `fossil_dispatch` are not part of the active v1.3 entry-point layer. Treat them as legacy code traces unless they are reactivated deliberately.

4.2 Time-Horizon and Rolling-Window Controls

In v1.3, temporal scope is defined in the entrypoint and then passed into preprocessing, optimization, and export routines.

Parameter in <code>main.py</code>	Unit	Meaning in v1.3
<code>periods</code>	time steps	Defines the active intra-annual time grid passed into preprocessing, optimization, and export layers.
<code>years</code>	model years	Defines the planning horizon over which policy, demand, transfer, capacity evolution, and sector-coupling decisions are evaluated.
<code>window</code>	years	Defines the number of planning years optimized in each rolling-horizon subproblem when <code>Rolling_Horizon=True</code> .

4.3 National Policy Vectors and General Cost Controls

Parameter in <code>main.py</code>	Unit	Structural role
<code>rps_National</code>	p.u.	Year-indexed national renewable-share path applied when <code>rps_inv_National=True</code> .
<code>CO2_Limit_National</code>	tCO2/year	Year-indexed national emissions ceiling applied when <code>carbonLimit_National=True</code> .
<code>ceep_parameter</code>	p.u.	Upper bound on annual electricity adequacy relaxation as a share of annual electric requirements.
<code>LostLoad_Cost</code>	EUR/MWh- eq.	Penalty assigned to explicit unmet-service variables.
<code>Transmission_Cost</code>	EUR/MW	Cost coefficient for endogenous transfer-capacity expansion.
<code>Import_Price</code>	EUR/MWh	Base price used for imported electricity.
<code>Imp_price_inc</code>	% per year	Annual escalation factor for import prices across model years.

4.4 Transport and Flexibility Assumptions

The current `main.py` keeps EV/V2G assumptions directly in the entrypoint, but their numerical values are case-specific and therefore omitted here. What matters structurally is the role of each parameter:

Parameter in <code>main.py</code>	Unit	Modeling meaning
<code>V2G_cost</code>	EUR/MWh	Marginal value or cost assigned to electricity returned from the EV fleet to the grid.
<code>ev_Demand_year</code>	demand scaling	Year-indexed EV demand scaling factor used to convert transport behavior into aggregate fleet load.
<code>ev_sto_min</code>	p.u.	Minimum allowable EV state-of-charge fraction.
<code>ev_Grid_eff</code>	p.u.	Charging/discharging efficiency applied in EV storage recursion.
<code>number_of_veh</code>	vehicles	Total fleet size represented in the aggregate EV model.
<code>average_ch_rate</code>	kW/vehicle	Typical charging power used to derive aggregate grid-connection capability.
<code>average_bat</code>	kWh/vehicle	Typical battery size used to derive aggregate EV storage energy.
<code>charging_veh</code>	MW	Derived aggregate EV charging power capability.
<code>stor_veh</code>	MWh	Derived aggregate EV storage-energy capability.
<code>ev_Grid_P</code>	MW by year	Year-indexed aggregate EV grid-exchange limit used in the power bounds.
<code>ev_stor</code>	MWh by year	Year-indexed aggregate EV storage-energy limit used in the SOC bounds.

4.5 Input Path Declarations in the Entrypoint

The active v1.3 entrypoint maps twelve numbered input objects into preprocessing:

Variable in main.py	File	Structural role
system_dat	1_system_data.xlsx	Defines the system layer and transfer-limit sheet used to construct regions and interconnections.
genco_dat	2_genco_data.csv	Master asset table for generation and technology identity.
demand_dat	3_demand.csv	Electricity-demand trajectories by period and year.
fuel_price_dat	4_fuel_cost.csv	Fuel-price and cost trajectories.
avl_factor_plant_dat	5_ncre_aval_factor.csv	Availability factors for non-dispatch renewable generation.
inflows_dat	6_scaled_inflows.csv	Hydro inflow trajectories.
import_export	7_import_export.csv	Transfer or import/export capacity references.
heat_demand_dat	8_heat_demand.csv	Heat-demand inputs by system and market.
cooling_demand_dat	9_cooling_demand.csv	Cooling-demand inputs by system and market.
ev_transpload_dat	10_ev_transp_load.csv	EV availability and transport-load profiles.
h2_demand_dat	11_demand_H2.csv	Hydrogen-demand trajectories.
flex_tech_dat	12_flex_tech.xlsx	Flexible-technology definitions, costs, COP values, and related structural tabs.

5 Input Data Engineering Guide

5.1 Why this is critical

For most new users, input preparation is the dominant workload. The model relies on joins across files using names and time indices; any mismatch can break preprocessing or invalidate economics.

5.2 Required Input Files and Roles

In the current v1.3 repository, the active reference packages are numbered multizone bundles such as `data/Colombia_Multizone` and `data/Colombia_Multizone_2050`. Structurally, every numbered file must be present, the two sheets inside `1_system_data.xlsx` must remain intact, and all names used across files must stay synchronized.

Input file	Primary role	Expected key fields	Main internal outputs	Critical consistency check
1_system_data.xlsx	System and transfer master workbook	System_Name ; transfer sheet with From , To , transfer capacity	systems , N_From , N_To , transfer_limits , link_check	system names must match suffixes used in demand/heat/cooling files and transfer endpoints must be valid system names
2_genco_data.csv	Core generation and technical master table	unit_name , system , fuel_type , technology , capacity, ramp, CO2, CHP fields	unit subsets by system, unit_system , max_load , cap_factor , CO2_factor	unit_name and system must align with availability, inflow, and technology files
3_demand.csv	Electricity demand by system/hour/year	year , period , system-level demand columns	demand	complete coverage of all (t, y) and system labels used in the model
4_fuel_cost.csv	Time-dependent fuel prices	year , period , fuel columns	var_cost , heat_var_cost , industry_fuels_price	fuel labels must match genco, thermal, and industry-fuel definitions
5_ncre_aval_factor.csv	Availability factors for non-dispatch units	year , period , one column per unit	aval_factor_plant	unit columns must match the non-dispatch unit names defined in 2_genco_data.csv
6_scaled_inflows.csv	Hydro inflow profiles	year , period , hydro-unit columns	hdam_inflow , hphs_inflow	inflow columns must match HDAM/HPHS unit names in the genco table
7_import_export.csv	Transfer and import/export reference series	year , period , transfer-related or interface-capacity fields	import/export auxiliary data and consistency checks	interface naming and time indices must align with the system and period definitions
8_heat_demand.csv	Heat demand for CHP markets and aggregate sectors	year , period , system-specific CHP and aggregate heat columns	heat_demand , Industry_HT_demand	CHP market names and system suffixes must align with CHP and heat-pump mappings
9_cooling_demand.csv	Cooling demand by system and market	year , period , system-specific cooling columns	cooling_demand	market names and system suffixes must align with the heat-pump structure
10_ev_transp_load.csv	EV availability and transport energy profile	year , period , EV profile columns	ev_availability , ev_transp_load	every modeled period must exist for each year; missing hours break EV recursion
11_demand_H2.csv	Hydrogen demand by sector/hour/year	year , period , hydrogen-demand sector columns	H2_demand	hydrogen-demand labels must match the sector logic used in preprocessing
12_flex_tech.xlsx	Flex assets, costs, COP, fuel shares, DH mapping	tabs for flexible technologies, thermal technologies, district heating, cost years, industry shares, COP	FC_* , HP_* , H2_* , Sta_* , HP_COP_period , fuel_share_max	sheet names, technology labels, and system mappings must remain synchronized across tabs

5.3 Interpreting the genco master table

The `2_genco_data.csv` file is the master unit table of the electricity-supply side in the current v1.3 repository. Its columns define the technical identity, cost structure, operational flexibility, reserve eligibility, storage behavior, CHP coupling, and long-term evolution of each generation asset represented in H2RES. In practical terms, this file acts as the central asset registry from which the preprocessing step builds unit subsets, cost parameters, emissions factors, operational limits, and intertemporal investment or retirement logic. Some fields describe short-term dispatch behavior, while others describe long-term capacity evolution under the scenario horizon. *v1.3 structural note: the genco table is now explicitly multizone through its system identifiers, so the same unit taxonomy must remain synchronized not only with availability and inflow files, but also with the system layer introduced by `1_system_data.xlsx`.*

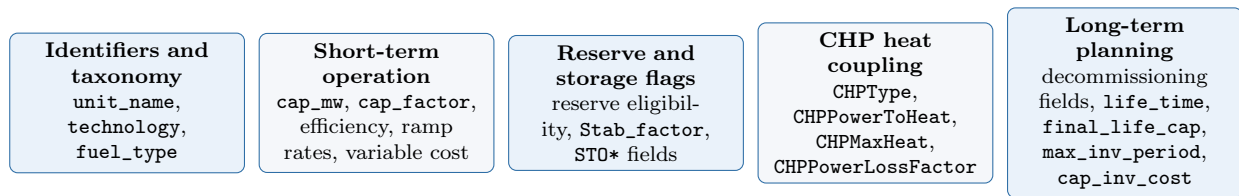


Figure 4: At-a-glance reading frame for the **genco** master table. The file mixes identifiers, operational data, storage/reserve fields, CHP coupling terms, and planning trajectories.

Local genco field	Unit	Meaning	Main role in H2RES
<code>unit_name</code>	-	Unique identifier of the generating unit.	Forms the master key used for model sets and for matching with time-series columns in availability, inflow, and some demand-linked files.
<code>cap_mw</code>	MW	Installed nominal power capacity.	Used to define <code>max_load_u</code> , generation limits, hydro conversion scales, and initial installed capacity.
<code>fuel_type</code>	-	Primary fuel classification of the unit.	Drives fuel-cost assignment, fuel-based subsets, and emissions accounting.
<code>decom_start_existing_cap</code>	year index	First modeled year at which the inherited capacity starts its retirement trajectory.	Used to build surviving existing-capacity terms for legacy assets.
<code>life_time</code>	year index	Last year used in the new-capacity survival calculation.	Combined with <code>decom_start_new</code> and <code>final_life_cap</code> to derive a geometric decommissioning rate for new builds.
<code>decom_start_new</code>	year index	First modeled year at which newly installed capacity begins its decommissioning trajectory.	Defines when new investments start to decay in the intertemporal capacity equations.
<code>final_life_cap</code>	p.u.	Residual fraction of capacity remaining at the end of the lifetime window.	Used in preprocessing to compute the annualized decommissioning rate of new capacity.
<code>max_inv_period</code>	MW	Maximum new capacity addition allowed in one modeled period.	Caps annual investment decisions for each unit.
<code>cap_factor</code>	p.u.	Capacity or utilization factor used in upper-bound logic.	Scales effective generation envelopes and annual production potential.
<code>efficiency</code>	p.u.	Conversion efficiency of the unit.	Used in variable-cost calculations and, for hydro/storage technologies, as discharge efficiency in storage dynamics.
<code>cost_no_fuel</code>	EUR/MW eq.	Non-fuel variable operating cost term.	Contributes to electricity-generation variable cost after fuel-price merging.
<code>cap_inv_cost</code>	EUR/MW	Capital cost of new investment.	Enters the capacity-expansion objective term.
<code>RampingCost</code>	EUR/MW	Cost coefficient for inter-hour output changes.	Used in ramping-cost linearization.

Local genco field	Unit	Meaning	Main role in H2RES
CO2Intensity	tCO2/MW	Direct carbon intensity of generation.	Used for CO2 accounting and carbon-cost terms.
technology	-	Technology label of the unit, such as HDAM, HPHS, HROR, PHOT, or thermal types.	Defines operational subsets and determines which constraint blocks apply to each unit.
RampUpRate	p.u./time step	Maximum upward change in output.	Used in dispatch ramp-up constraints.
RampDownRate	p.u./time step	Maximum downward change in output.	Used in dispatch ramp-down constraints.
PrimaryReserve	flag	Eligibility marker for primary reserve provision.	Determines whether a unit enters the primary-reserve eligible set.
SecondaryReserve	flag	Eligibility marker for secondary reserve provision.	Determines whether a unit enters the secondary-reserve eligible set.
Stab_factor	p.u.	Stability or reserve-scaling factor for reserve feasibility.	Used in reserve-down feasibility relationships.
CHPType	flag	CHP activation/type indicator in the local schema.	Determines whether CHP-specific coupling and heat-storage parameters are read for the unit.
STOCapacity	MWh or MWh _{th}	Storage capacity associated with hydro or CHP-linked storage.	Used as hydro reservoir energy limit or CHP thermal storage capacity, depending on the unit type.
STOSelfDischarge	p.u./day	Storage self-discharge or daily heat-loss rate.	Used for CHP thermal-storage losses and interpreted consistently with storage-type logic.
STOMaxChargingPower	MW	Maximum charging power for storage-capable units.	Defines the upper charging limit for technologies that can absorb electricity into storage.
STOChargingEfficiency	p.u.	Charging efficiency for storage-capable units.	Determines how much stored energy is retained from each unit of charging input.
CHPPowerToHeat	p.u.	Power-to-heat coupling ratio of CHP operation.	Converts extracted electric-equivalent input into useful heat in CHP constraints.
CHPPowerLossFactor	p.u.	Electricity-output loss associated with heat extraction.	Reduces feasible electric generation when CHP heat is produced.
CHPMaxHeat	MW _{th}	Maximum heat-production capability of the CHP unit.	Bounds CHP heat output in coupled heat-power operation.

Important distinction: the `genco` file mixes short-term operational parameters with long-term planning parameters. Fields such as `decom_start_existing_cap`, `decom_start_new`, `life_time`, `final_life_cap`, and `max_inv_period` are not dispatch inputs; they are scenario-planning parameters used by the local preprocessing and investment model.

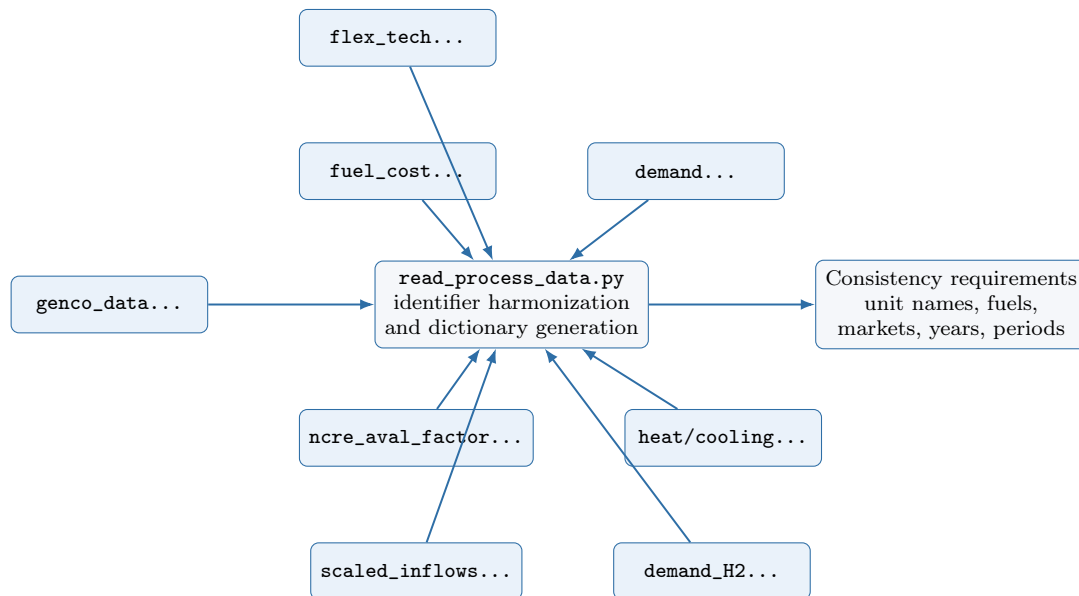


Figure 5: Cross-file consistency network required before model construction.

5.4 How Wide-Format Files Become Long-Format Tables

Several core files arrive in a wide format, where each technology or unit appears as a separate column. During preprocessing, those files are reshaped into long-format tables keyed by **year**, **period**, and an identifier such as **unit_name** or fuel label. For new users, this transformation is one of the most important ideas behind the matching rules presented in this section. *v1.1 update: the current preprocessing script performs these reshaping steps from CSV and XLSX inputs using Pandas directly rather than XML tree parsing.*

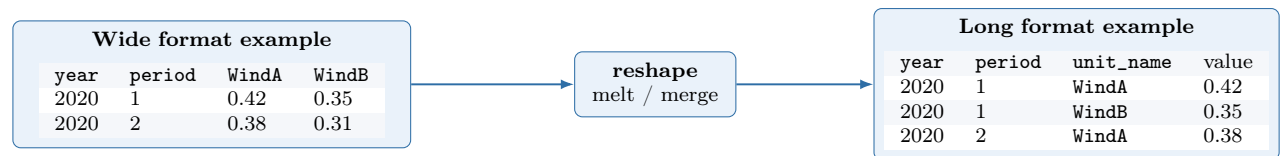


Figure 6: Conceptual wide-to-long transformation used for files such as renewable availability and hydro inflows. Matching then occurs on **year**, **period**, and the reshaped identifier column.

5.5 Cross-file Consistency Rules

1. **Unit-name integrity:** columns used as unit identifiers across inflow, availability, and technology files must be identical strings. In this repository, **unit_name** from **genco_data_HR_sdewes.csv** is the reference key.
2. **Time-grid integrity:** all time-dependent files should share the same year set and full period set.
3. **Fuel-label integrity:** fuel labels must align across genco, thermal, fuel-cost, and industry tables.
4. **Heat/cooling market integrity:** CHP heat-market names and DH mapping labels must be consistent.
5. **Technology-key integrity:** technology names in **flex_tech** must also exist in **tech_cost_years** and, where applicable, in **COP_HP**.

Mandatory matching map

The table below summarizes the minimum identifier matches that must hold across files before running the model. This is the most important practical checklist for avoiding preprocessing failures, silent data loss in merges, or incorrectly parameterized units.

Reference file/key	Must match with	Matching field(s)	Match type	Why it matters / failure mode
genco_data_HR_sdewes.csv : unit_name	ncre_aval_factor_HR_2020_2050_sdewes.csv column headers	unit_name , plus year , period	exact string match	Required for assigning availability factors to non-dispatchable units. If the name does not match, the renewable or HROR time series will not merge correctly into aval_factor_plant_{u,t,y} .

Reference file/key	Must match with	Matching field(s)	Match type	Why it matters / failure mode
genco_data_HR_sdwes.csv : unit_name of HDAM/HPHS units	scaled_inflows_HR_2020_2050_sdwes.csv column headers	unit_name, plus year, period	exact string match	Required for assigning inflows to hydro reservoir and pumped-hydro units. A mismatch prevents the hydro time series from being attached to the intended units.
genco_data_HR_sdwes.csv : fuel_type	fuel_cost_2020_2050_sdwes.csv column headers	fuel_type, plus year, period	exact label match	Required for variable-cost calculation. The code explicitly stops if a unit cannot find a matching fuel-price series.
genco_data_HR_sdwes.csv : unit_name where CHPType=='Y'	heat_demand_HR_2020_2050_sdwes.csv column headers	unit_name, plus year, period	exact string match	CHP units are linked to heat-demand columns by name. If the CHP unit name is absent or inconsistent, the heat-demand assignment becomes invalid.
cooling_demand_HR_2020_2050_sdwes.csv market labels	heat-pump and DH market logic from flex_tech_HR_2020_2050_sdwes.xlsx	market names, plus year, period	exact string match	Cooling balances rely on consistent market naming. A mismatch disconnects cooling demand from the technologies meant to serve it.
demand_2020_2050_sdwes.csv sector columns	internal electricity-demand sector set	demand-sector names, plus year, period	schema match	The electricity-demand table is melted into the internal sector set. Missing or altered sector columns break or distort the demand dictionary.
demand_H2_2020_2050_sdwes.csv sector columns	internal hydrogen-demand sector set	demand-sector names, plus year, period	schema match	Hydrogen demand is built by melting sector columns. Missing sector names lead to incomplete $H2_demand_{s,t,y}$ coverage.
heat_demand_HR_2020_2050_sdwes.csv : Industry_HT	flex_tech_HR_2020_2050_sdwes.xlsx sheet Industry	shared industrial-demand logic by year, period, fuel labels	semantic match	The industrial heat-demand trajectory and the industrial fuel-switching table must refer to the same industrial block. Inconsistency distorts fuel-switching bounds and hydrogen substitution.
flex_tech_HR_2020_2050_sdwes.xlsx : sheet Industry Fuel_source	fuel_cost_2020_2050_sdwes.csv column headers	fuel label, plus year, period	exact label match	Used to build industry fuel prices and endogenous fuel shares. A mismatch means the corresponding industrial fuel cannot be priced correctly.
flex_tech_HR_2020_2050_sdwes.xlsx : sheet Industry Fuel_source	same sheet CO2_factor rows and internal industry_fuels set	fuel label	exact label match	Required so each industrial fuel has both price and CO2 intensity for fuel-switching and emissions accounting.
flex_tech_HR_2020_2050_sdwes.xlsx : sheet flex_tech unit_name	sheet tech_cost_years unit_name	unit_name, year	exact string match	Cost trajectories for flex technologies assume synchronized technology names across sheets. A mismatch breaks CAPEX/OPEX assignment by technology-year pair.
flex_tech_HR_2020_2050_sdwes.xlsx : sheet flex_tech heat-pump names	sheet COP_HP columns	heat-pump identifier, plus year, period	exact string match	Required to generate $HP_COP_period_{h,t,y}$. If the COP table does not match the heat-pump names, the conversion efficiency path is incomplete.

Reference file/key	Must match with	Matching field(s)	Match type	Why it matters / failure mode
import_export_HR_2020_2050_sdewes.csv	all time-series files	period ; where applicable year	time-grid match	Import bounds are applied by period. Missing or inconsistent periods create inconsistent balance equations across datasets.
All time-series files	model horizon	year , period coverage	completeness match	Every active model year-period pair must exist in demand, H2 demand, heat, cooling, EV, inflow, and availability inputs. Missing rows cause partial dictionaries and undefined behavior in constraint construction.

In the local preprocessing code, both [scaled_inflows_HR_2020_2050_sdewes.csv](#) and [ncre_availability_factor_HR_2020_2050_sdewes.csv](#) are first reshaped into a long format with the key columns **year**, **period**, and **unit_name**, and then merged with the **genco** table. As a consequence, hydro inflow columns must exactly match the **unit_name** of HDAM and HPHS units, and renewable availability columns must exactly match the **unit_name** of the non-dispatchable generation units that use those profiles. Any mismatch in spelling, capitalization, whitespace, or naming convention breaks that join and prevents the corresponding time series from being assigned correctly.

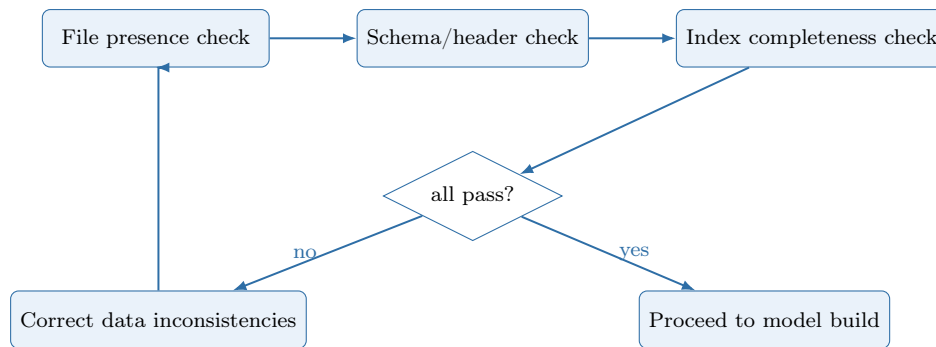


Figure 7: Recommended pre-run validation logic for robust scenario execution.

5.6 Pre-run Data Validation Checklist

- Are all required files present and correctly referenced in `main_rolling_horizon_v3.py`?
- Are all years present in every time-dependent file?
- Are all periods complete within each year?
- Are there duplicated records for the same identifier tuple?
- Are there missing values in key fields (**unit_name**, **year**, **period**, **capacity**, **efficiency**)?
- Are scenario switches compatible with available data (reserves, hydrogen, heat pumps)?

6 How H2RES Represents the Energy System

This section is written for first-time users who need to understand the logic of the model before working through traceability tables and equation appendices. The objective is not to restate the full mathematical formulation, but to show how data, physical flows, and decision variables connect across the main subsystems.

6.1 Integrated system view

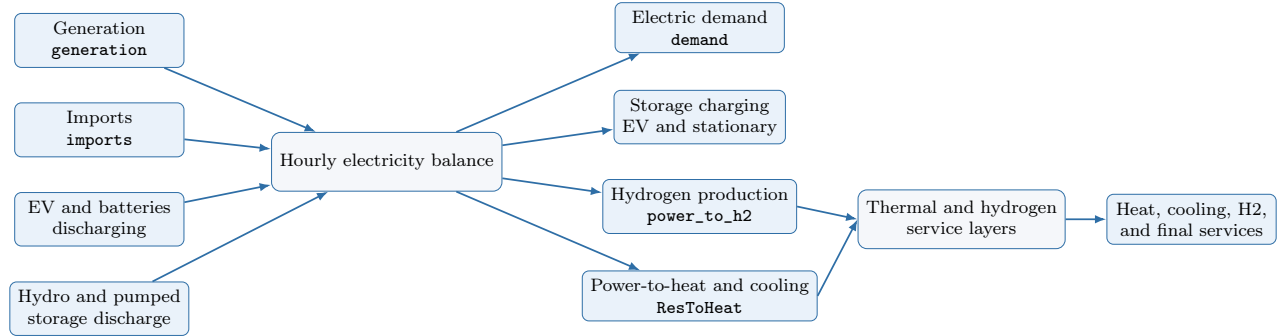


Figure 8: Integrated system view of the main carrier interactions represented in H2RES.

Purpose: give a first-pass view of how electricity sits at the center of the model and feeds storage, hydrogen, and heat/cooling service layers.

Read it as: the left-hand side contains hourly electric injections, the center enforces the electricity balance, and the right-hand side shows where electricity is consumed directly or transformed into other energy carriers.

Key variables: $\text{generation}_{u,t,y}$, $\text{imports}_{t,y}$, $\text{power_to_h2}_{t,y}$, $\text{ResToHeat}_{t,y}$, $\text{ev_sto_in/out}_{t,y}$, $\text{stat_sto_in/out}_{b,t,y}$.

Linked model blocks: generation and investment, electricity balance, EV storage, stationary batteries, hydrogen and fuel cells, heat/cooling and CHP.

6.2 How input data becomes model structure

Purpose: show that H2RES is built through preprocessing joins, not by reading one monolithic table.

Read it as: the raw files on the left are harmonized in `read_process_data.py`; the joins only succeed when identifiers and time axes are consistent.

Key identifiers: `unit_name`, `fuel_type`, `year`, and `period`. These keys determine whether operational profiles, costs, and technical attributes are assigned to the correct asset or demand block.

Linked manual sections: Input Data Engineering Guide, Cross-file Consistency Rules, and Appendix D.

6.3 Electricity balance logic

Purpose: make clear that H2RES is organized around one central adequacy equation at every hour and model year.

Read it as: all electricity injections must be matched by direct electric demand, storage charging, conversion to hydrogen or heat, and any residual adequacy slack represented by `ceep`.

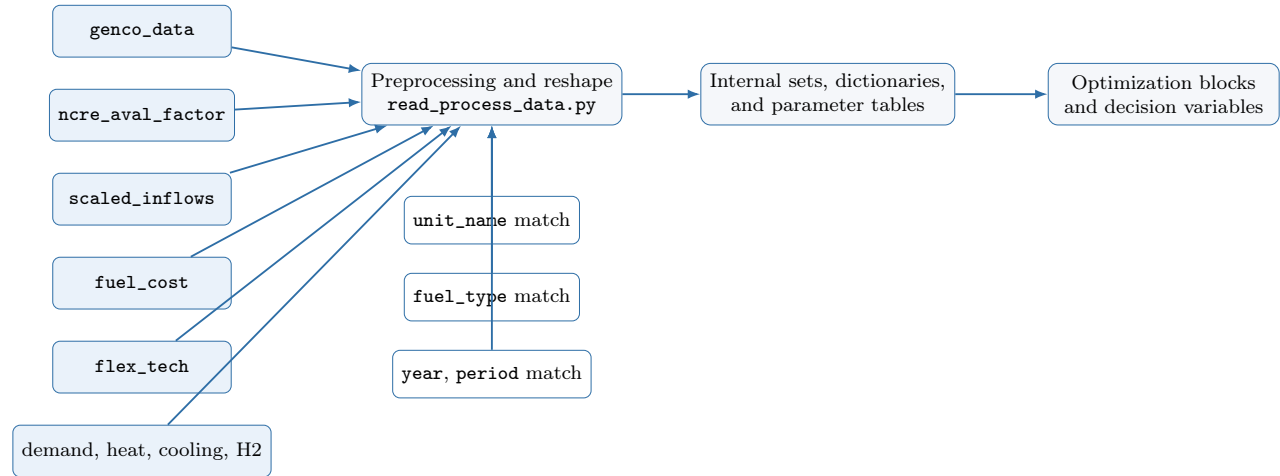


Figure 9: Data-engineering logic from source files to model-ready parameters and sets.

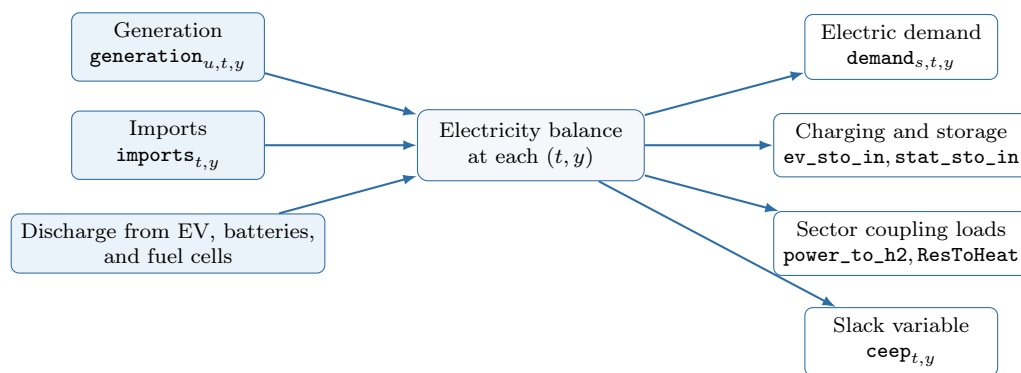


Figure 10: Conceptual reading of the hourly electricity balance in H2RES.

Key variables: $\text{generation}_{u,t,y}$, $\text{imports}_{t,y}$, $\text{ev_sto_out}_{t,y}$, and $\text{stat_sto_out}_{b,t,y}$ define the main hourly injections, while $\text{power_to_h2}_{t,y}$, $\text{ResToHeat}_{t,y}$, and $\text{ceep}_{t,y}$ sit on the withdrawal side of the balance.

Why this matters for new users: many apparent anomalies in outputs are not solver issues; they are usually consequences of how charging loads, hydrogen production, or heat electrification appear on the right-hand side of this balance.

6.4 Heat and cooling service logic

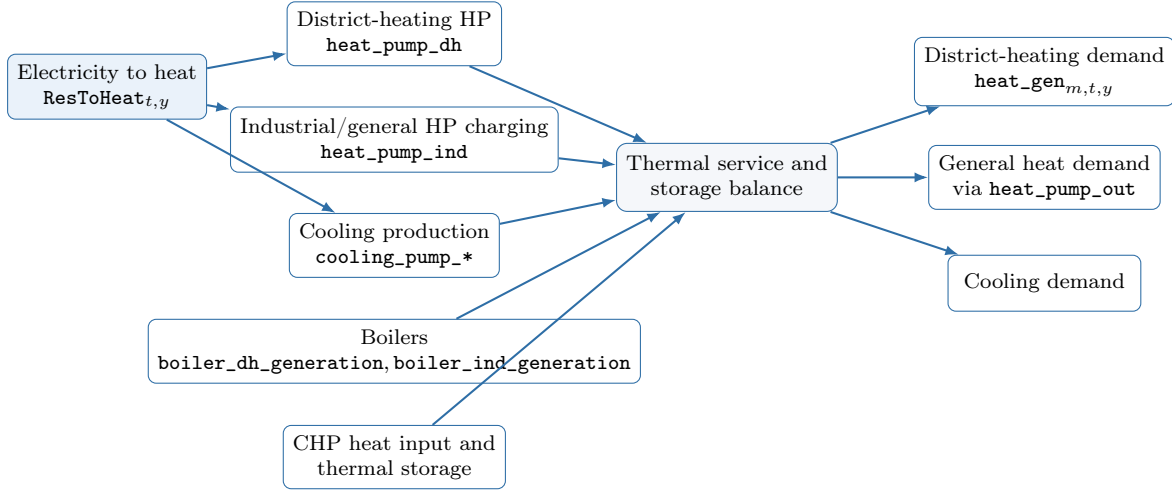


Figure 11: Heat and cooling service logic, including heat pumps, boilers, CHP, and thermal storage.

Purpose: clarify the most confusing part of the model for new users: electricity does not serve heat demand directly, but through heat-pump conversion, CHP coupling, boiler support, and thermal storage release.

Read it as: ResToHeat is the aggregate electric input to thermal conversion; that input is distributed across district-heating heat pumps, industrial or general heat pumps, and cooling devices, then combined with boilers and CHP-linked heat to satisfy thermal services.

Frequently confused variables:

Variable	Unit	Practical meaning for a new user
$\text{heat_pump_ind}_{h,t,y}$	MW_{th}	Heat added into the industrial/general thermal-storage pathway by a heat pump before final service is released.
$\text{heat_pump_out}_{h,t,y}$	MW_{th}	Heat withdrawn from that heat-pump thermal pathway to satisfy general heat demand.
$\text{heat_pump_sto_soc}_{h,t,y}$	MWh_{th}	State of charge of the heat-pump-related thermal buffer over time.
$\text{heat_gen}_{m,t,y}$	MW_{th}	District-heating service delivered by CHP-linked thermal systems to meet market-specific heat demand.
$\text{boiler_ind_generation}_{b,t,y}$	MW_{th}	Boiler heat sent directly to the non-district industrial/general heat service branch.

Linked model blocks: heat/cooling and CHP, industrial fuel switching, and the thermal parts of Appendix C.

6.5 Hydrogen subsystem logic

Purpose: show that hydrogen in H2RES is not an isolated demand block; it is an intertemporal energy carrier connected back to electricity through fuel cells.

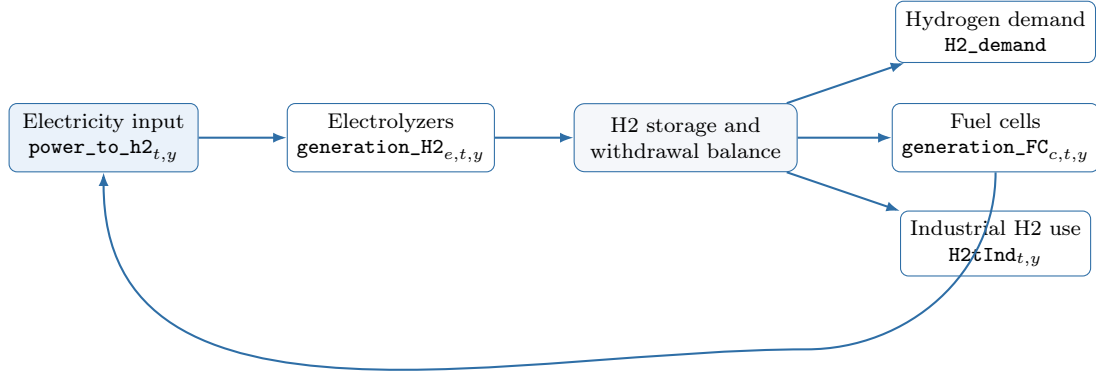


Figure 12: Hydrogen subsystem from electric conversion to storage, end use, and fuel-cell return.

Read it as: electricity first feeds electrolysis, hydrogen can then be stored, and stored hydrogen is allocated among exogenous hydrogen demand, industrial substitution, and electricity re-generation through fuel cells.

Key variables: $\text{power_to_h2}_{t,y}$ is the electric load used for hydrogen conversion; $\text{generation_H2}_{e,t,y}$, $\text{h2_sto_soc}_{s,t,y}$, and $\text{h2_sto_out}_{s,t,y}$ describe hydrogen production and inventory; $\text{generation_FC}_{c,t,y}$ and $\text{H2tInd}_{t,y}$ capture the main downstream uses.

Why this matters for new users: high electricity demand from electrolysis can be entirely consistent with low direct electric demand growth, because the hydrogen branch is represented as an additional conversion load rather than a conventional demand sector.

6.6 Storage logic across carriers

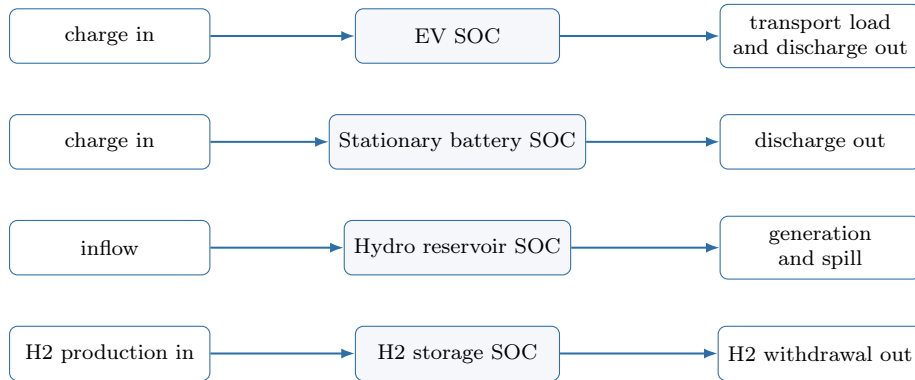


Figure 13: Common state-of-charge logic across EV, stationary battery, hydro, and hydrogen storage.

Purpose: help new users recognize that several subsystems differ in physics but share the same intertemporal modeling pattern: previous inventory plus inflow minus outflow, subject to capacity and terminal conditions.

Read it as: each storage family has its own carrier and constraints, but all of them are modeled as state variables coupled across hours and years.

Key state variables: $\text{ev_sto_soc}_{t,y}$, $\text{stat_sto_soc}_{b,t,y}$, $\text{storage_level}_{u,t,y}$, $\text{h2_sto_soc}_{s,t,y}$.

Practical implication: when a scenario changes storage size, flexibility, or inflow assumptions, the effect is usually spread over multiple hours rather than appearing as a single-period response.

7 Adapting H2RES to a New Case Study

This section translates common user tasks into the files and assumptions that must be changed. It is intentionally task-oriented rather than file-oriented, because new users usually start from a research question, not from the internal repository structure.

7.1 What usually needs to be changed

Task	Main input block to revise	Priority	Why it matters	Typical failure if wrong
Change the scenario name, switches, or policy path	Scenario controls in <code>main_rolling_horizon_v3.py</code>	Mandatory	Defines the active scenario, optional blocks, and core policy assumptions for the run.	Wrong module activation, overwritten result folders, or unintended policy behavior.
Change electricity demand trajectories	Electricity demand data	Mandatory for a new load case	Alters the central electricity balance and adequacy requirements.	Missing periods, distorted load levels, or infeasibility.
Change generation asset portfolio	Generation master table	Mandatory when units differ	Defines the set of units, their technical attributes, and their classification into model subsets.	Missing units in availability or inflow mappings, wrong technology subsets, or inconsistent costs.
Change renewable availability profiles	Renewable availability profiles	Mandatory for new RES units or profiles	Controls non-dispatchable hourly production envelopes.	Unit-name mismatch or silent misassignment of hourly profiles.
Change hydro inflows	Hydro inflow profiles	Mandatory for new hydro cases	Drives reservoir and pumped-hydro intertemporal availability.	Hydro units without inflow series or incorrect water-energy behavior.
Change import or export assumptions	Interconnection and trade assumptions	Case-dependent	Alters interconnection limits and the import-bound interconnection envelope used by the active v1.1 balance formulation.	Unrealistic adequacy support or broken export logic.
Change fuel prices	Fuel price data	Usually mandatory	Affects dispatch costs, industrial-fuel economics, and thermal operating cost.	Fuel-label mismatch or distorted merit-order outcomes.
Change heat, cooling, or hydrogen demand	Sector-coupling demand data	Mandatory for sector-coupling studies	Changes service obligations in thermal and hydrogen blocks.	Missing market labels, broken closure constraints, or implausible coupling results.
Change EV, H2, storage, heat-pump, or boiler technology assumptions	Flexibility technology data and <code>main_rolling_horizon_v3.py</code>	Usually mandatory for flexibility studies	Controls technology costs, efficiencies, limits, DH mapping, and industrial fuel shares.	Technology-key mismatch across sheets or inconsistent bounds in coupled sectors.
Change technology learning or surviving-capacity trajectories	Intertemporal trajectories in <code>main_rolling_horizon_v3.py</code>	Optional but often important	Governs intertemporal evolution of investment attractiveness and inherited stock.	Scenario narrative inconsistent with long-term capacity behavior.

Task	Main input block to revise	Priority	Why it matters	Typical failure if wrong
Change time-horizon or rolling-window assumptions	Temporal controls in <code>main_rolling_horizon_v3.py</code>	Mandatory when altering solve strategy	Changes the number of time steps, the planning years optimized together, and whether the run is full-horizon or rolling-horizon.	Inconsistent runtime expectations, incomparable scenarios, or misleading interpretation of intertemporal investment behavior.

7.2 Recommended order when adapting the model

1. Change only high-level scenario settings in `main_rolling_horizon_v3.py` and rerun the baseline data package.
2. Replace or update core demand and supply master tables.
3. Update profile files that must match those master tables by identifier and time grid.
4. Update sector-coupling workbooks (`flex_tech`, heat, cooling, hydrogen) only after core identifiers are stable.
5. Re-run the validation logic in Sections 5, 8, 10, and Appendix D before interpreting any new scenario result.

7.3 What should usually *not* be changed first

- Do not start by modifying `Build_model.py`; most new-case adaptations belong to inputs or `main_rolling_horizon_v3.py`, not to equations.
- Do not rename units, fuels, or markets in only one file; identifier changes must propagate through every matching file.
- Do not tighten several policy trajectories simultaneously before a relaxed case runs correctly.

8 Traceability: Input Data to Model Blocks

8.1 Block-Level Traceability Matrix

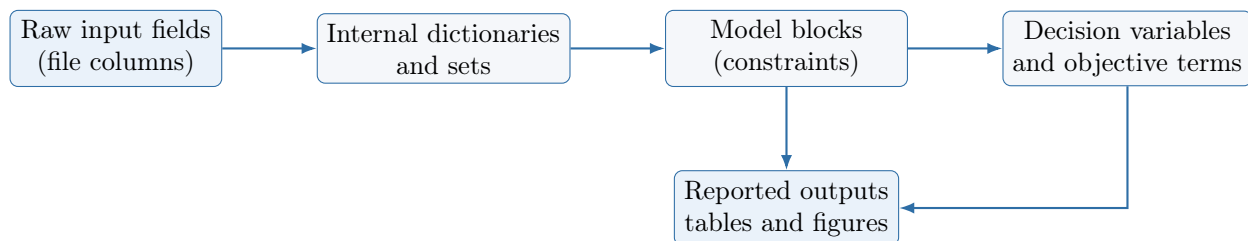


Figure 14: Traceability chain from source data to optimization outputs.

Model block	Primary data sources	Representative internal parameters	Main decision variables	Main constraints in code
Generation and investment	genco, fuel cost, availability	max_load, cap_factor, var_cost, decom_rate	generation, cap_inv	max_output*, max_inv_unit
Electricity balance and imports	demand, import-export, EV/storage coupling	demand, import_ntc	imports, ceep	meet_demand, import_lim
Hydrogen and fuel cells	flex_tech, H2 demand, tech-cost-years	H2_eff, H2_demand, FC_eff	generation_H2, h2_sto_soc, generation_FC	Power_to_h2, h2_storage*, max_output_FCell
EV and stationary storage	EV profile, flex_tech, tech-cost-years	ev_availability, Sta_sto_eff, Sta_sto_max_cap	ev_sto_*, stat_sto_*	ev_storage*, stat_storage*
Heat/cooling and CHP	heat demand, cooling demand, flex_tech, CHP fields in genco	HP_COP_period, CHPPowerToHeat, STOCapacity_heat	heat_pump_*, boiler_*, heat_gen, heat_sto_level	heat_pump*, meet_heat_demand*, meet_cooling_demand*
Hydro storage dynamics	inflow file + genco hydro fields	hdam_inflow, hphs_inflow, storage capacities	storage_level, sto_out_flow	storage_level*, hphs_storage*
Policies and emissions	main policy vectors, genco/thermal/industry CO2 data	rps, CO2_limit, CO2_factor, Fuel_CO2	CO2_total*, all operational vars	rps, CO2_emissions*
Primary/secondary reserves (optional)	reserve flags in genco/flex_tech	reserve unit sets, reserve_factor	reserve up/down variables by technology	primary_*, secondary_* constraint families

8.2 Critical Parameter Traceability Examples

Internal parameter	Source file	Source field(s)	Dimensions	Why it matters
var_cost	fuel cost + genco	fuel price, cost_no_fuel, efficiency	(u, t, y)	Directly drives dispatch merit order and objective cost.
HP_COP_period	flex_tech (COP_HP)	HP columns by year, period	(h, t, y)	Converts electrical input into heat/cooling output.
fuel_share_max	flex_tech (Industry) + fuel prices + CO2 price	Industry_HT, fuel prices, CO2_factor	(t, y, f)	Governs endogenous industrial fuel switching.
import_ntc	import-export file	Import_capacity	(t)	Caps import dependence and affects adequacy margins.
TechChange	main_rolling_horizon_v3.py vectors + unit filters	TechChangeSolar, TechChangeWind	(u, y) for wind/solar	Alters intertemporal attractiveness of RES investments.

8.3 Comprehensive Parameter Hand-off Table

This table expands the parameter traceability requested for implementation-level work. It focuses on parameters explicitly created/processed in `read_process_data.py` and consumed in `Build_model.py`.

Internal parameter (model notation)	Source file/sheet	Source fields	Indices in model	Unit	Main consuming blocks
$demand_{s,t,y}$	demand_2020_2050_sdewes.csv	sector columns, year , period	(s, t, y)	MW_e	Electricity balance, total load, adequacy and CEEP bounds
$H2_demand_{s,t,y}$	demand_H2_2020_2050_sdewes.csv	sector columns, year , period	(s, t, y)	MW_{H2}	Hydrogen withdrawal balance and reserve feasibility margins
$heat_demand_{m,t,y}$	heat_demand_HR_2020_2050_sdewes.csv	market columns, general_demand , year , period	(m, t, y)	MW_{th}	CHP heat balance, district heating closure, general heat adequacy
$cooling_demand_{m,t,y}$	cooling_demand_HR_2020_2050_sdewes.csv	market columns, general_cooling , year , period	(m, t, y)	MW_{cool}	Cooling adequacy constraints in heat-pump/cooling block
$var_cost_{u,t,y}$	genco + fuel_cost_2020_2050_sdewes.csv	cost_no_fuel , fuel prices, efficiency	(u, t, y)	EUR/MWh _e	Objective operating cost for electricity generation
$heat_var_cost_{b,t,y}$	thermal_tech + fuel cost	fuel price, thermal efficiency	(b, t, y)	EUR/MWh _{th}	Objective operating cost for thermal boilers
$industry_fuels_price_{f,t,y}$	fuel_cost_2020_2050_sdewes.csv	fuel columns by time	(f, t, y)	EUR/MWh _{fuel}	Industrial fuel expenditure term in objective
$fuel_share_max_{t,y,f}$	flex_tech... sheet Industry + fuel cost + CO2 price	Industry_HT, CO2_factor, fuel price	(t, y, f)	p.u.	Industrial fuel-switch constraint upper bounds
$import_ntc_t$	import_export_HR_2020_2050_sdewes.csv	Import_capacity	(t)	MW_e	Import upper bound and adequacy flexibility
$exports_t$	import_export_HR_2020_2050_sdewes.csv	total_exports	(t)	MW_e	Legacy interconnection series retained in the input schema; the active v1.1 base-line formulation no longer depends on a separate export-aware balance switch
max_load_u	genco_data_HR_sdewes.csv	cap_mw	(u)	MW_e	Capacity envelopes, reserves, ramping, hydro energy conversion
cap_factor_u	genco_data_HR_sdewes.csv	cap_factor	(u)	p.u.	Dispatch upper bounds for thermal/hydro and CHP extraction logic
$cap_inv_cost_u$	genco_data_HR_sdewes.csv	cap_inv_cost	(u)	EUR/MW	Capacity expansion CAPEX in objective
$ramp_up_rate_u$, $ramp_down_rate_u$	genco_data_HR_sdewes.csv	RampUpRate, RampDownRate	(u)	p.u./h	Inter-hour generation ramp constraints
$CO2_factor_u$	genco_data_HR_sdewes.csv	CO2Intensity	(u)	tCO2/MWh _e	Power-sector annual emissions accounting and carbon cost
$H2_eff_e$	flex_tech... sheet	electrolyzer efficiency	(e)	p.u.	Power-to-hydrogen conversion, reserve feasibility for electrolyzers
FC_eff_c	flex_tech... sheet	fuel-cell efficiency	(c)	p.u.	Fuel-cell generation and hydrogen-use coupling
$HP_COP_period_{h,t,y}$	flex_tech... sheet	COP_HP	(h, t, y)	p.u.	Electric-to-heat conversion and reserve flexibility for HP
$Sta_sto_eff_b$	flex_tech... sheet	storage efficiency	(b)	p.u.	Stationary battery SOC recursion and electric balance terms
$h2_sto_init_s$, $H2_sto_max_cap_s$	flex_tech... sheet	cap_mw, max_cap for H2 storage	(s)	MWh_{H2}	Hydrogen SOC bounds and storage expansion limit
$hdam_inflow_{u,t,y}$, $hphs_inflow_{u,t,y}$	scaled_inflows_HR_2020_2050_sdewes.csv	unit inflow columns + time keys	(u, t, y)	p.u. inflow	Reservoir/pumped-hydro dynamic balance and availability

Internal parameter (model notation)	Source file/sheet	Source fields	Indices in model	Unit	Main consuming blocks
$\text{aval_factor_plant}_{u,t,y}$	ncre_aval_factor_HR_2020_2050_sdeves.csv	plant availability columns + time keys	(u, t, y)	p.u.	Equality dispatch for non-dispatchable units (wind/solar/HROR)
$\text{TechChange}_{u,y}$	<code>main_rolling_horizon_v3.py</code> + unit filters from genco	<code>TechChangeSolar</code> , <code>TechChangeWind</code>	(u, y)	p.u. factor	Technology-specific investment attractiveness in objective
NPV_y	<code>main_rolling_horizon_v3.py</code>	discount vector	(y)	p.u.	All intertemporal objective components
rps_y	<code>main_rolling_horizon_v3.py</code>	annual policy trajectory	(y)	p.u.	Renewable portfolio share policy constraint
$\text{CO2_limit}_y, \text{CO2_price}_y$	<code>main_rolling_horizon_v3.py</code>	annual policy trajectories	(y)	tCO ₂ /year, EUR/tCO ₂	Emissions cap constraints and carbon-cost terms
reserve_factor_u	genco reserve flags (PrimaryReserve)	<code>Stab_factor</code>	(u)	p.u.	Reserve-down technical feasibility for dispatchable units
$\text{ev_availability}_{t,y}$, $\text{ev_transp_load}_{t,y}$, ev_Grid_P_max_y , ev_sto_max_y	ev_transp_load.csv + <code>main_rolling_horizon_v3.py</code>	profile columns + EV assumptions	$(t, y), (y)$	p.u., MW _e , MWh _e	EV SOC recursion, charging/discharging limits, electricity balance

8.4 Decision Variables Organized by Model Block

The matrix below complements Section 3 by grouping variables according to the operational blocks used in `Build_model.py`.

Model block	Main decision variables (original notation)	Main indices	Functional role
Generation and investment	$\text{generation}_{u,t,y}$, $\text{cap_inv}_{u,y}$, $\text{imports}_{t,y}$, $\text{ceep}_{t,y}$	(u, t, y) , (u, y) , (t, y)	Core electric dispatch and capacity expansion with adequacy slack
Hydrogen and fuel cells	$\text{generation_H2}_{e,t,y}$, $\text{h2_sto_soc}_{s,t,y}$, $\text{h2_sto_out}_{s,t,y}$, $\text{generation_FC}_{c,t,y}$, $\text{cap_inv_FC}_{c,y}$, $\text{h2_electrolysis_max_cap}_{e,y}$	(e, t, y) , (s, t, y) , (c, t, y) , (c, y) , (e, y)	Couples electricity, hydrogen conversion, storage, and re-electrification
EV storage	$\text{ev_sto_in}_{t,y}$, $\text{ev_sto_out}_{t,y}$, $\text{ev_sto_soc}_{t,y}$	(t, y)	Transport electrification flexibility through managed charging/discharging
Stationary batteries	$\text{stat_sto_in}_{b,t,y}$, $\text{stat_sto_out}_{b,t,y}$, $\text{stat_sto_soc}_{b,t,y}$, $\text{stat_sto_cap_inv}_{b,y}$	(b, t, y) , (b, y)	Short-term balancing and intertemporal storage capacity expansion
Heat/cooling and CHP	$\text{heat_pump_dh}_{h,m,t,y}$, $\text{heat_pump_ind}_{h,t,y}$, $\text{cooling_pump_dh}_{h,m,t,y}$, $\text{cooling_pump_ind}_{h,t,y}$, $\text{heat_pump_out}_{h,t,y}$, $\text{heat_pump_sto_soc}_{h,t,y}$, $\text{boiler_dh_generation}_{b,m,t,y}$, $\text{boiler_ind_generation}_{b,t,y}$, $\text{heat_gen}_{m,t,y}$, $\text{heat_sto_level}_{m,t,y}$	(h, m, t, y) , (h, t, y) , (b, m, t, y) , (b, t, y) , (m, t, y)	Closes thermal and cooling balances and links them to electricity via COP/CHP coupling
Industrial fuel switching	$\text{Fuel_ind}_{f,t,y}$, $\text{H2tInd}_{t,y}$	(f, t, y) , (t, y)	Endogenous industrial fuel-mix allocation and hydrogen substitution
Hydro reservoirs and pumped hydro	$\text{storage_level}_{u,t,y}$, $\text{sto_out_flow}_{u,t,y}$, $\text{generation}_{u,t,y}$	(u, t, y)	Intertemporal water-energy accounting with spill and terminal constraints
Reserves (optional)	$\text{primary_*.}_{t,y}$, $\text{secondary_*.}_{t,y}$, technology-specific reserve variables for units/storage/FC/ELY/HP	$(\cdot, t, y)$	Up/down balancing capability under primary and secondary reserve modules
Emissions accounting and policy	CO2_total_y , CO2_total_heat_y , $\text{CO2_total_industry}_y$	(y)	Tracks sectoral annual emissions and enforces cap trajectories

8.5 Unit Convention and Dimensional Consistency

H2RES is solved on an hourly time grid. For this reason, many balance equations use hourly-average power variables that are numerically equivalent to energy over one hour. To avoid ambiguity in publication-grade analysis, use the convention below consistently.

Quantity family	Unit convention	Practical interpretation	Validation rule
Electric generation and flows (generation , imports , ceep , storage power)	MW_e (hourly average)	Average electric power in one model period (1 hour).	In each hourly electricity balance, all additive terms must use MW_e -equivalent magnitudes.
Electric storage state variables (ev_sto_soc , stat_sto_soc , hydro storage_level)	MWh_e	Stored electric energy at period boundary.	SOC recursion must combine previous SOC and hourly inflow/outflow power terms consistently over 1-hour step.
Heat and cooling flows	MW_{th} , MW_{cool}	Hourly thermal/cooling service rate.	Heat/cooling demand closure constraints must be unit-homogeneous at each (t, y) .
Hydrogen production and storage	MW_{H2} , MWh_{H2}	Hydrogen power-rate equivalent and inventory.	Conversion constraints must apply $H2_{eff}/FC_{eff}$ to map electric and hydrogen domains.
Costs and prices	EUR, EUR/MWh, EUR/MW, EUR/tCO2	Objective components under annual discounting (NPV).	Every objective term must be reduced to EUR before aggregation.
Emissions	tCO2, tCO2/MWh	Sectoral annual emissions and intensities.	Annual cap uses consistent sum: power + heat + industry emissions in tCO2/year.
Dimensionless controls (rps , reserve factors, availability, COP, efficiencies)	p.u.	Fractions or coefficients.	Must multiply physically compatible quantities; never add directly to power/energy terms.

8.6 Embedded Model Traceability Within This Manual

Model traceability in this manual is organized around three complementary layers:

- The main body explains the user workflow, data flow, system logic, traceability, and validation sequence.
- Appendix C maps selected implemented constraint groups to their mathematical meaning and interpretation.
- Appendix D documents the data-matching logic required for preprocessing consistency.

Together, these components provide a concise link between model formulation, implementation structure, and input-data consistency.

9 Interpreting Outputs

9.1 Where results are written

Outputs are written to:

```
results/<scenario_name>/
```

where `<scenario_name>` is the value of `scen` in `main.py`.

In the bundled v1.3 workflow, `export_plot_multizone_RH.py` removes and recreates this folder, then writes multizone rolling-horizon figures and optional workbooks. The auxiliary script `save_all_results_to_csv.py` writes low-level CSV traces under `results/<scenario_name>/data_result/`.

9.2 Main output families

- Region-level generation-by-fuel figures and optional workbooks.
- Regional hydro-storage trajectories and hydro-related exports.

- Inter-system import/export flow outputs derived from `imports_from_results`.
- Regional industrial-fuel and hydrogen-use outputs.
- Regional and aggregate CO2-emissions summaries.
- Low-level CSV traces for investment, dispatch, storage, unmet service, and transfer variables when the CSV export utility is executed.

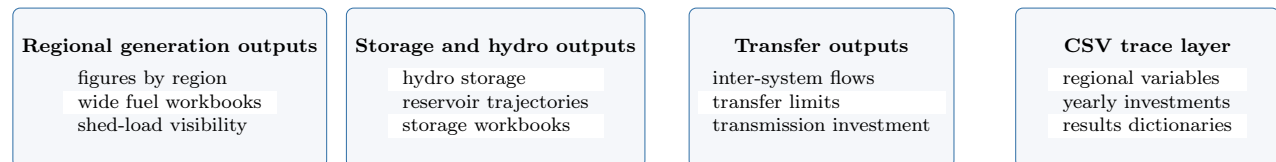


Figure 15: Main output families in the active v1.3 multizone export stack.

9.3 Recommended reading order for outputs

1. Confirm that the results directory corresponds to the intended scenario name.
2. Read the region-level generation outputs to understand the broad dispatch pattern first.
3. Check hydro, storage, and transfer outputs to understand how balancing is achieved spatially and temporally.
4. Review industrial-fuel, hydrogen, and CO2 outputs to connect sector coupling with policy outcomes.
5. Use the `data_result` CSV layer for audit-level validation or custom post-processing.

10 Troubleshooting Matrix

Issue pattern	Likely cause	Suggested check	Corrective action
Model infeasible	Inconsistent demand/capacity/policy assumptions	Inspect solver status prints and tightest policy settings	Start from relaxed policy test case, then re-tighten incrementally
KeyError during model build	Missing index tuples in dictionaries	Check completeness of (t, y) and (u, t, y) combinations after preprocessing	Fill missing rows; enforce complete hourly grids
Fuel price stop in preprocessing	Fuel label mismatch between files	Compare <code>fuel_type</code> labels with fuel-cost columns	Harmonize labels or add missing fuel columns
Heat/cooling imbalance behavior	Market-name mismatch or missing general demand columns	Verify <code>general_demand</code> , <code>general_cooling</code> , CHP market names	Align labels across heat/cooling and DH mapping files

Issue pattern	Likely cause	Suggested check	Corrective action
Reserve module errors	Reserve switches active with incomplete reserve-eligible sets	Check Y/N flags and resulting reserve sets from preprocessing	Populate reserve flags or disable reserve switches for baseline run
Outputs overwritten	Same scenario name reused	Check active exporter folder-reset behavior	Archive old run or use a new <code>scen</code> label
Numeric issues status	Poor data scaling or extreme parameter values	Inspect magnitude ranges for costs, capacities, and flows	Recheck units and scale assumptions consistently

11 Verification and Validation (V&V) Protocol

11.1 Structural Validation Before Solve

Apply these checks prior to model construction:

1. Identifier integrity across files (`unit_name`, fuel labels, market labels, technology keys).
2. Full time-grid coverage for all (t, y) required by active modules.
3. No duplicate tuples for primary keys in melted long-format tables.
4. No nulls in mandatory fields used to build dictionaries.
5. Scenario-switch compatibility with data availability (reserves, hydrogen, heat-pump blocks).

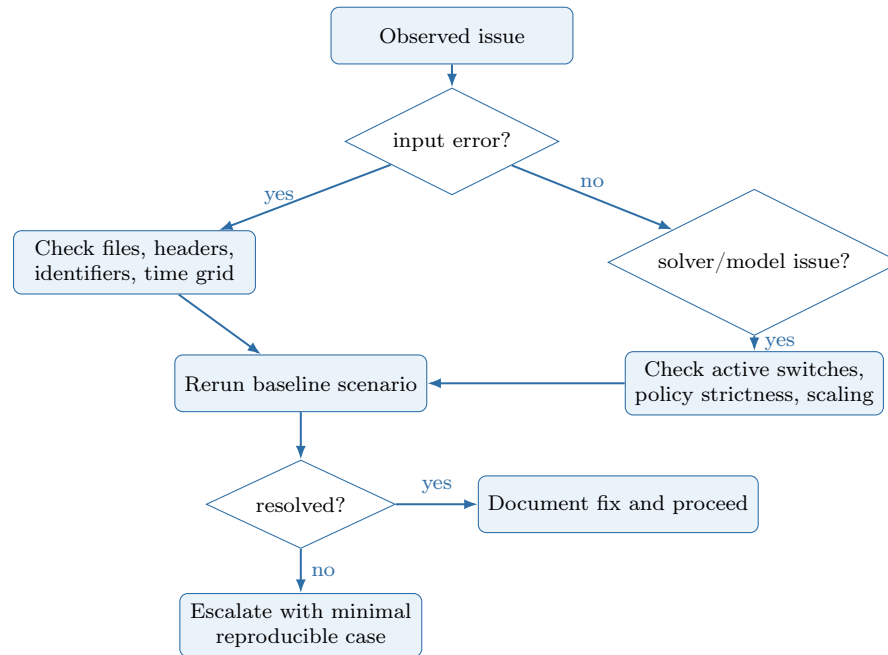


Figure 16: Troubleshooting decision path for rapid diagnosis and recovery.

11.2 Numerical and Behavioral Validation After Solve

Validation target	Computation approach	Acceptance criterion	Action if failed
Hourly electricity balance residual	Recompute LHS-RHS for each (t, y) .	Absolute residual $\leq 10^{-6}$ in model units.	Inspect indexing completeness, sign convention, and optional module activation logic.
Hydrogen subsystem residual	Check production-storage-demand consistency for each (t, y) .	Absolute residual $\leq 10^{-6}$.	Recheck <code>H2_eff</code> , demand indexing, and storage capacity dictionaries.
Heat/cooling demand closure	Recompute heat and cooling equalities by market and period.	Residual near solver tolerance.	Verify DH mapping, <code>general_demand</code> , and COP table completeness.
Policy compliance flags	Evaluate RPS and CO2 constraints against reported totals.	Constraints respected when active; unconstrained when disabled.	Verify boolean switches and policy vectors in <code>main.py</code> .
Intertemporal SOC sanity	Inspect SOC trajectories and terminal conditions.	No negative inventory if constrained; terminal rules satisfied.	Recheck SOC bounds and decomposition factors.

11.3 Scenario Toggle Regression Tests

For robust research workflows, keep a minimal regression matrix:

- Baseline: reserves off, moderate policy.
- +Heat/H2 active.
- +Primary reserve active.
- +Secondary reserve active.
- Tight policy stress test (higher RPS and lower CO2 cap).

Track solver status and key outputs across the matrix to detect unintended behavioral drift after data/code changes.

12 PhD-Level Research Practice and Reproducibility

12.1 Minimum reproducibility package per scenario

For each scenario run, store:

- Scenario metadata: all switch values, policy vectors, and input-path declarations from `main.py`.
- Input dataset snapshot (immutable copy or versioned hash).
- Solver version and parameter settings.
- Output folder and post-processing scripts used.
- Brief run log describing unusual behavior or manual data corrections.

12.2 Reproducible Baseline Mini-Case (Bundled Multizone Template)

Use the default repository setup as a reproducibility anchor:

1. Set baseline scenario switches in `main.py` and record them in the run log.
2. Run `python main.py` from repository root.
3. Confirm solver status prints `Status==2` or, only with explicit justification from the solver log, `Status==13`.
4. Verify output folder creation under `results/<scenario>/`.
5. Confirm presence of core artifacts: regional generation outputs, storage files, transfer outputs, CO2 summaries, and the optional `data_result` CSV layer.
6. Recompute one-hour checks for electricity and hydrogen balances from exported tables.

Expected baseline behavior:

- No hard infeasibility for default data package.
- Policy constraints bind only when their switches are enabled.
- Module-specific outputs are null/absent when corresponding modules are disabled.

12.3 Recommended experimental protocol

1. Validate baseline data with all optional modules off (reserves off, moderate policy).
2. Activate one complexity layer at a time (heat/hydrogen/reserves).
3. Perform sensitivity analysis on a small set of policy and cost drivers.
4. Report both system-level outputs and binding constraints to explain causality.

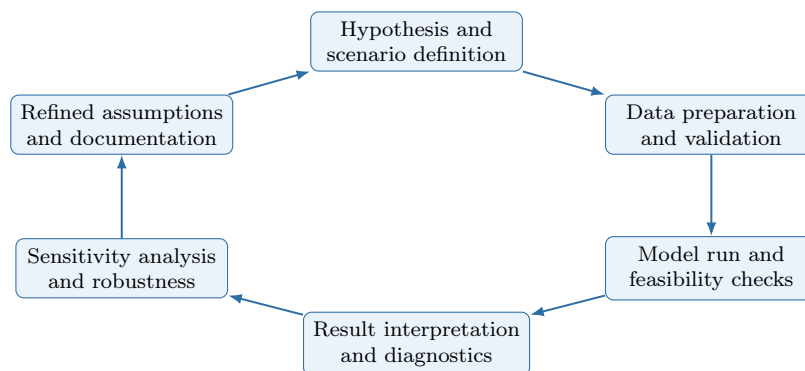


Figure 17: Recommended research cycle using H2RES for transparent iteration.

12.4 When to modify equations

Equation modifications should start only after:

- data consistency checks pass,
- baseline scenario reproduces expected behavior,
- traceability links (input $\rightarrow$ parameter $\rightarrow$ constraint) are documented.

13 Modeling Assumptions, Implications, and Risks by Block

Model block	Core assumption	Implication for results	Main risk if violated
Electricity adequacy	All active loads and couplings are represented in hourly balance.	Dispatch and imports are interpreted as physically feasible hourly system operation.	Hidden or mis-indexed load terms can create false adequacy or false deficits.
Hydrogen subsystem	Efficiency-based conversion and storage recursions are complete.	H2 expansion and FC dispatch are economically comparable with direct electrification pathways.	Mis-specified H2 efficiencies can bias both cost and feasibility.

Model block	Core assumption	Implication for results	Main risk if violated
Heat/cooling subsystem	COP and CHP coupling correctly map electric-thermal interactions.	Heat and cooling service are co-optimized with electricity and fuel switching.	Inconsistent market labels or COP profiles can distort heat-electrification conclusions.
Reservoir and pumped hydro	Inflows and storage bounds represent physical water availability.	Hydro dispatch captures intertemporal opportunity cost of water.	Inflow misalignment by unit/time can produce unrealistic hydro flexibility.
Reserves	Eligible sets and reserve factors represent technology capability.	Reserve costs and capability margins shape flexibility investment and operation.	Incorrect reserve eligibility can overstate balancing security.
Policy blocks	RPS/CO2/CEEP constraints reflect intended trajectories and switches.	Policy sensitivity studies are interpretable and reproducible.	Mis-specified policy vectors lead to misleading transition pathways.
Costing and discounting	Objective terms are monetized consistently in EUR using NPV.	Intertemporal trade-offs are coherent across CAPEX/OPEX/policy costs.	Unit mismatch in costs can invert investment rankings.

14 Known Boundaries and Open Items

- Some legacy variable names in code are not self-explanatory and can complicate first-time interpretation.
- Environment bootstrapping commands (dependency installation specification) are not formalized in a repository lockfile.
- Traceability is intentionally summarized at block and selected-constraint level; a future code-auditing appendix could expand this to a full line-by-line implementation map if required.

A Flex-Technology Workbook Sheet Roles

Worksheet	Core fields	Role in model
<code>flex_tech</code>	<code>unit_name</code> , <code>Tech_type</code> , efficiency, capacities, reserve flags	Defines FC, electrolyzers, HP, H2 storage, and stationary storage sets and parameters.
<code>thermal_tech</code>	thermal unit fields, efficiency, CO2, decomposition fields	Defines thermal boiler technologies for heat blocks.
<code>DistrictHeating</code>	<code>unit</code> + DH-market columns	Maps initial DH capacities by technology-market pair.
<code>tech_cost_years</code>	<code>year</code> + technology columns	Provides annualized CAPEX trajectories for flexible technologies.
<code>Industry</code>	<code>Fuel_source</code> , <code>Industry_HT</code> , <code>CO2_factor</code>	Drives industrial fuel-share and fuel-switch logic.
<code>COP_HP</code>	<code>year</code> , <code>period</code> , HP columns	Provides hourly COP profiles used in power-to-heat conversion.

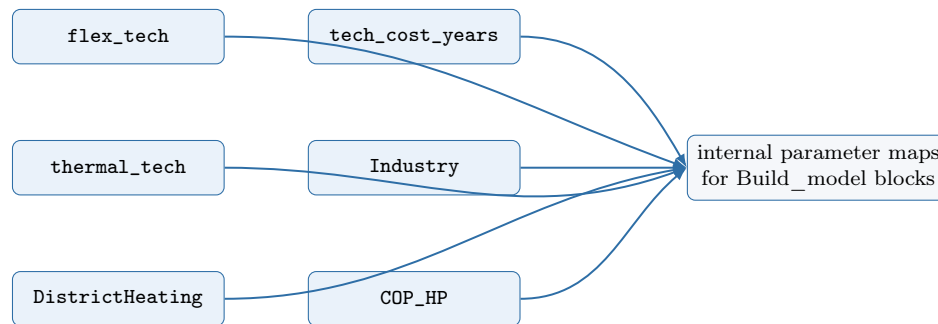


Figure 18: How flex-technology workbook sheets feed model parameter dictionaries.

B Suggested Future Validation Script Logic

An optional `validate_input_data.py` could run:

1. file-existence checks,
2. required column checks per file/sheet,
3. duplicate-key checks (`year, period, ...`),
4. missing-value checks in mandatory fields,
5. cross-file name matching (units, fuels, markets, technologies),
6. time-grid completeness checks.

This should remain non-invasive (read-only checks, no data mutation).

C Equation Traceability from `Build_model.py` with User Commentary

This appendix provides an equation-level interpretation of representative implemented constraints.

C.1 Code-to-Equation Mapping (selected core constraints)

Code reference	Constraint/object in code	Simplified equation form	Commentary for user interpretation
L70-L72	<code>no_inv_resToheat,</code> <code>no_resToheat,</code> <code>no_heat_pump</code>	If <code>NoResToHeatInv=True</code> , force <code>heat_pump_cap = ResToHeat = heat_pump_out = 0</code> .	This switch does not relax heat equations; it removes one pathway (power-to-heat), so heat must be met by remaining technologies.
L239-L242	<code>ind_demand_meet,</code> <code>industry_demand_meet</code>	Fuel-share cap plus demand closure: $\sum_f \text{Fuel_ind}_{f,t,y} + \text{H2tInd}_{t,y} = \text{Industry_HT_demand}_{t,y}$.	Industrial heat is always met; the optimization chooses fuel split within allowed shares and prices.
L247	<code>Power_to_h2</code>	Electrolyzer output divided by efficiency equals electric input for hydrogen production.	Electricity consumed by electrolysis is explicitly linked to hydrogen output and cannot be hidden elsewhere.
L255-L264	<code>h2_storage_*</code>	Hydrogen SOC recursion with upper/lower/terminal bounds.	Hydrogen can shift energy across hours, but bounded inventory prevents unrealistic unlimited shifting.
L284-L300	<code>ev_storage_*</code>	EV SOC recursion with transport withdrawal and charging efficiency; charge/discharge power limited by availability.	EV flexibility is conditional on parking/availability profile, not a free battery.
L305-L323	<code>stat_storage_*</code>	Stationary battery SOC recursion and investment-dependent SOC cap.	New storage capacity expands feasible SOC envelope over years.
L382 or L363	<code>meet_demand</code> or <code>meet_demand_export</code>	Hourly electric injections = withdrawals + coupling loads + slack.	This is the central adequacy equation; most infeasibilities propagate here when inputs are inconsistent.
L401	<code>ceep_demand</code>	Annual $\sum_t \text{ceep}_{t,y}$ bounded by <code>ceep_parameter</code> times annual electric needs.	Reliability slack exists but is policy-limited; tighter cap can make scenarios infeasible.
L414-L443	<code>heat_pump_*</code> , <code>heat_pump_soc_*</code>	COP-based electricity-to-heat relation and thermal SOC dynamics for heat pumps.	Clarifies why <code>heat_pump_ind</code> and <code>heat_pump_out</code> differ: one charges thermal storage and the other serves demand-side release.
L467-L500	CHP/boiler/heat demand constraints	CHP extraction coupling, boiler caps, heat storage balance, heat and cooling demand closure.	Thermal demand is met through multiple parallel pathways; this block defines how those pathways interact hour-by-hour.
L515-L553	HDAM/HPHS storage constraints	Reservoir SOC bounds, inflow-driven recursion, generation/spill feasibility, terminal condition.	Prevents end-horizon depletion and enforces physically feasible hydro trajectories.
L560-L567	ramping and imports	Inter-hour ramp limits and import smoothing ($\pm 20\%$ step) with NTC cap.	Controls operational realism and avoids abrupt imports or generator swings.
L571, L587-L602	<code>rps,</code> <code>CO2_emissions_*</code>	Renewable share minimum and annual emissions cap decomposition.	Portfolio and carbon policy are enforced after dispatch and sector-coupling interactions are accounted for.
L608-L612	<code>ramp_up/down_cost</code>	Auxiliary ramp cost variables lower-bound absolute generation changes (split sign-wise).	Adds economic penalty for fast dispatch variation while preserving linearity.
L652-L673	<code>objective</code>	Z = sum of all discounted OPEX/CAPEX/policy terms listed in the objective.	Confirms exactly which terms are monetized; useful when diagnosing unexpected investment or dispatch choices.

C.2 How to Read This Traceability Appendix

This appendix intentionally avoids reproducing long raw code listings. Instead, it provides a compact interpretation layer between the implemented model and the mathematical logic described earlier in the manual. The intended reading order is:

1. use Sections 4 and 5 to understand the model blocks conceptually;
2. use Appendix C.1 to identify how representative implemented constraints correspond to those blocks; and

3. use Appendix D to verify that the required input mappings and identifiers are consistent enough for those constraints to be meaningful.

In other words, Appendix C is the implementation-oriented interpretation layer of this PDF, while Appendix D is the data-consistency layer that supports it.

D Complete Data Matching and Consistency Map

This appendix expands the core matching guidance from Section 3 into a more exhaustive audit map. The purpose is to distinguish between:

1. **hard joins in code**, where inconsistent keys directly break preprocessing or leave required values undefined;
2. **required schema fields**, where missing identifiers or column names make downstream construction impossible; and
3. **semantic consistency checks**, where the code may still run but the scenario interpretation becomes unreliable.

D.1 Hard Joins in Code

Source table / field	Must match with	Join keys	Join type in preprocessing	Consequence if inconsistent
hydro <code>genco.unit_name</code>	<code>scaled_inflows</code> columns	<code>year</code> , <code>period</code> , <code>unit_name</code> after melting inflow columns	explicit merge after wide-to-long transformation	Hydro inflow series are not assigned to the intended storage units; reservoir equations then operate with incomplete or missing inflow data.
non-dispatch <code>genco.unit_name</code>	<code>ncre_aval_factor</code> columns	<code>year</code> , <code>period</code> , <code>unit_name</code> after melting availability columns	explicit merge after wide-to-long transformation	Renewable availability is not linked to the intended unit; generation limits become wrong or missing for the affected assets.
<code>genco.fuel_type</code>	<code>fuel_cost</code> columns	<code>year</code> , <code>period</code> , <code>fuel_type</code>	left merge of long fuel-price table into <code>genco</code> time expansion	Variable generation cost cannot be built correctly; in the current preprocessing logic, missing values trigger a hard stop for affected units.
<code>thermal_tech.fuel_type</code>	<code>fuel_cost</code> columns	<code>year</code> , <code>period</code> , <code>fuel_type</code>	left merge into thermal technology expansion	Boiler and thermal-heat variable costs become undefined or inconsistent with the intended fuel assumptions.
<code>Industry.Fuel_source</code>	industry labels + <code>fuel_cost</code> columns	<code>Fuel_source</code> , then <code>year</code> , <code>period</code> , <code>Fuel_source</code>	merge of industrial-fuel metadata with long fuel-switching table	Industrial fuel-switching costs and emissions factors cannot be assembled consistently; some fuels may silently drop from the switching block.
<code>tech_cost_years</code> technology columns	<code>flex_tech.unit_name</code>	<code>year</code> , <code>unit_name</code> after melting technology columns	explicit merge to create annual CAPEX trajectories	Flex technologies receive no annual cost trajectory, so investment terms for those technologies become undefined or mismatched.
<code>COP_HP</code> technology columns	heat-pump identifiers in <code>flex_tech</code>	<code>year</code> , <code>period</code> , <code>variable</code> where <code>variable</code> must correspond to the heat-pump identifier	explicit merge after wide-to-long transformation	Heat-pump COP values are not linked to the correct technology, which distorts electricity-to-heat conversion and reserve/flexibility logic.

Source table / field	Must match with	Join keys	Join type in pre-processing	Consequence if inconsistent
DistrictHeating.unit	DH market columns and linked equipment names	unit plus melted market identifier	explicit melt and restructuring into DH-capacity dictionaries	District-heating heat pumps and boilers are not mapped to the intended market/unit pairing, causing incorrect initial capacities and feasible heat supply structure.
CHP-linked heat_demand columns	CHP genco.unit_name	year, period, unit_name after melting demand columns	explicit merge-like alignment through long-format demand construction	CHP heat balance is attached to the wrong market label or not attached at all, invalidating thermal demand satisfaction.
cooling-demand market columns	cooling-service identifiers in the HP block	year, period, unit_name-style market naming after melt	long-format alignment into cooling demand dictionaries	Cooling demand may remain disconnected from heat-pump cooling variables, so the model solves a different cooling problem than intended.
CO2_price.year	industrial fuel-switching rows	year	left merge on annual table	Carbon-price signals are not propagated to industrial switching costs, causing incorrect cost comparison across fuels.
import_export.period	model periods set	period	direct alignment by period index	Import capacity and export trajectories become incomplete relative to the dispatch horizon.
ev_transp_load.year, period	model years and periods	year, period	direct alignment into EV availability and transport-load dictionaries	EV charging/discharging feasibility is built on an incomplete or shifted time grid.

D.2 Required Schema Columns and Identifier Fields

File / sheet	Required identifiers or columns	Category	Why they are structurally required
genco_data_*.csv	unit_name, technology, fuel_type, cap_mw, efficiency, cost_no_fuel, cap_inv_cost, CO2Intensity	master plant schema	This is the master table for generation assets. The model partitions units by technology, attaches time series by unit_name, and derives dispatch, cost, emissions, reserve, and lifetime behavior from these fields.
genco_data_*.csv	RampUpRate, RampDownRate, RampingCost	operational realism	These columns parameterize ramp constraints and ramping penalties in <code>Build_model.py</code> . Missing values weaken or invalidate operational flexibility representation.
genco_data_*.csv	PrimaryReserve, SecondaryReserve, Stab_factor	reserve eligibility	These fields determine reserve participation and reserve-down scaling for generation units. Missing or inconsistent values distort reserve feasibility.
genco_data_*.csv	CHPType, CHPPowerToHeat, CHPPowerLossFactor, CHPMaxHeat, STOCapacity, STOSelfDischarge	CHP and heat coupling	CHP heat-production limits and thermal storage dynamics depend on these identifiers. If a CHP unit exists without the matching CHP fields, the heat block becomes underdefined.
genco_data_*.csv	decom_start_existing_cap, decom_start_new, life_time, final_life_cap, max_inv_period	long-term planning	These fields feed decommissioning and investment-envelope logic. Missing values break the time evolution of effective capacity.
flex_tech sheet	unit_name, Tech_type, efficiency, cap_mw, max_cap, var_cost	flex technology master schema	These identify each non-genco flexibility asset and define its technical and economic envelope.
flex_tech sheet	decom_start_new, life_time, final_life_cap, heat_storage_cap	long-term flex evolution	These support investment, degradation/survival logic, and heat-storage coupling for heat pumps and storage technologies.
flex_tech sheet	PrimaryReserve, SecondaryReserve	reserve eligibility	These fields identify which flex technologies are allowed to provide reserve services.
thermal_tech sheet	unit_name, fuel_type, cap_mw, max_cap	thermal heat schema	These columns define boiler technologies and link them to fuel-price and emissions information.

File / sheet	Required identifiers or columns	Category	Why they are structurally required
tech_cost_years sheet	year plus one column per flex technology	annual CAPEX trajectories	The sheet is melted into (year, unit_name) cost rows. Missing technology columns mean the corresponding flex asset has no investment-cost path.
COP_HP sheet	year, period plus one column per heat-pump technology	hourly COP trajectories	The heat-pump electricity-to-heat relation is indexed on both year and period. The time grid must therefore exist explicitly.
DistrictHeating sheet	unit plus market-specific columns	DH market mapping	This sheet maps district-heating technologies to market labels and initial capacities used in the thermal block.
Industry sheet	Fuel_source, Industry_HT, CO2_factor	industrial switching schema	These columns define candidate industrial fuels, the industrial high-temperature demand context, and the fuel-level emissions factors.
scaled_inflows	year, period plus one column per hydro storage unit	time series input	Hydro inflow assignment depends on a complete time grid and exact unit-name matches after melt.
ncre_aval_factor	year, period plus one column per renewable unit	time series input	Renewable generation profiles are created from this file; mismatched columns leave units without availability factors.
fuel_cost	year, period plus one column per fuel	time series input	All fuel-based variable costs depend on this table after wide-to-long conversion.
heat_demand	year, period, CHP market columns, general_demand, Industry_HT	demand schema	The heat block expects both district-heating and general-demand labels; the industrial demand logic also expects the industrial high-temperature label to exist.
cooling_demand	year, period, general_cooling and any other cooling-market columns	demand schema	Cooling balances are built from these labels and must exist with the same time grid as the heat block.
import_export	period, Import_capacity, total_exports	interconnection schema	Import limits and exports are consumed directly by the electricity-balance logic.
ev_transp_load	year, period, ev_availability, ev_transp_load	EV time series schema	The EV block requires both grid-availability and transport-withdrawal series on the full model time grid.

D.3 Semantic Consistency Checks That May Not Trigger Immediate Errors

Consistency check	Files / objects involved	Why the run may still complete	Why the result can still be wrong
Technology taxonomy alignment	genco.technology, flex_tech.Tech_type, manual scenario switches in main_rolling_horizon_v3.py	Labels can be syntactically present even if they are not used consistently across modules.	Units may be assigned to the wrong model subset, so constraints are applied to the wrong assets or omitted entirely.
Fuel taxonomy alignment	genco.fuel_type, thermal_tech.fuel_type, Industry.Fuel_source, fuel-cost columns	Generic strings can match partially or be interpreted inconsistently without an immediate parser failure.	Electricity, heat, and industry can end up using conceptually different fuels that appear similar by name, corrupting cost and emissions comparisons.
Year-grid consistency	main_rolling_horizon_v3.py year lists and all annual or hourly input files	The code can process files with overlapping but not identical year domains.	Scenario trajectories, decommissioning vectors, CAPEX paths, and policy series can become misaligned across years.
Period-grid consistency	all hourly/time-slice files	A file may load even if some periods are missing or ordered differently.	Demand, inflow, availability, COP, transport load, and import limits may refer to different hourly structures.
District-heating market semantics	DistrictHeating, heat_demand, CHP and boiler market labels	Labels can exist in both places but represent different aggregation logic.	Heat is allocated to the wrong district-heating market, so feasible thermal service is misrepresented.
Cooling market semantics	cooling_demand, heat-pump cooling variables	Cooling labels may be present without a hard-coded failure.	Cooling demand can be solved against the wrong service bucket or omitted from the intended technology pathway.

Consistency check	Files / objects involved	Why the run may still complete	Why the result can still be wrong
Industrial high-temperature demand semantics	<code>heat_demand</code> , <code>Industry</code> sheet, industrial fuel-switching logic	The model can build the industrial switching block if identifiers exist, even if the demand interpretation is inconsistent.	Hydrogen substitution and industrial fuel switching may be benchmarked against the wrong thermal demand level.
Reserve switch coherence	<code>main_rolling_horizon_v3.py</code> switches, reserve flags in <code>genco</code> and <code>flex_tech</code>	A reserve module can activate even when eligibility flags are poorly calibrated.	Reserve shortages or artificial reserve abundance can appear because participating units were not tagged consistently.
EV parameter interpretation	<code>main_rolling_horizon_v3.py</code> EV/V2G assumptions and <code>ev_transp_load</code> time series	Both sources can be valid syntactically.	The aggregate battery size, power limits, and transport withdrawals may correspond to incompatible fleet assumptions.
Currency convention alignment	user manual convention, scenario comments in <code>main_rolling_horizon_v3.py</code> , input cost files	Legacy comments do not stop execution.	Users may compare or calibrate monetary inputs inconsistently if some annotations still suggest USD while the model documentation assumes EUR.
Unit scaling conventions	MW vs. MWh-per-hour interpretations across demand, storage, and inflow files	Numerically similar scales can pass through the pipeline.	Scenario interpretation becomes wrong if one file is treated as hourly power while another is implicitly energy-per-step.

D.4 Recommended Pre-Run Review Logic

Before launching a new scenario, the user should verify the following in order:

1. **Identifier audit:** every required `unit_name`, `fuel_type`, market label, and technology label appears exactly as expected across all linked files.
2. **Time-grid audit:** all hourly inputs share the same `year` and `period` structure.
3. **Schema audit:** every mandatory column listed in Section D.2 exists before preprocessing starts.
4. **Semantic audit:** market labels such as `general_demand`, `general_cooling`, and `Industry_HT` are interpreted consistently across all relevant files.
5. **Scenario audit:** annual vectors in `main_rolling_horizon_v3.py` use the same horizon and ordering as the data files they are intended to parameterize.

This appendix is intended to support a future automated validator, but it is already sufficient as a manual audit checklist for dataset preparation.